
Live-Sensorchoreographie: Entwicklung eines visuellen Programmier-Interfaces für die Live-Orchestrierung kollaborativer audiovisueller Smartphone-Performances

Masterarbeit zur Erlangung des akademischen Grades
Master of Science
im Studiengang Medieninformatik
an der Fakultät für Informatik und Ingenieurwissenschaften
der Technischen Hochschule Köln

vorgelegt von: Leander Gerwing
Matrikel-Nr.: 11145833
Adresse: Brückenstraße 34
51643 Gummersbach
leander_robert_bernhard.gerwing@smail.th-koeln.de

eingereicht bei: Prof. Dr. Christian Noss
Zweitgutachter*in: Prof. Dr. Stefan Bente

Gummersbach, 20.04.2026

Kurzfassung/ *Abstract*

Diese Arbeit stellt Leitmotiv (Gerwing, 2026) vor, ein webbasiertes Werkzeug zur Erstellung von kollaborativen und partizipativen audiovisuellen Inhalten über ein visuelles Programmierinterface. Ermöglicht wird dies durch die Nutzung von WebRTC für eine beinahe-Echtzeitkommunikation und modernen Browser-Schnittstellen, zum Beispiel zur Nutzung von Sensoren wie Beschleunigungssensoren oder Gyroskopen von Smartphones.

Es wird die vollständige Durchführung der Entwicklung dieses Tools beschrieben. Dies umfasst den Designprozess, die Implementation und die Evaluation.

Keywords: visual programming, participative art, collaborations in live performances, ui design, web development, live coding, webrtc

Inhaltsverzeichnis

Tabellenverzeichnis	IV
Abbildungsverzeichnis	V
1 Einleitung	1
1.1 Forschungsfrage	2
1.2 Zielsetzung	2
1.3 Struktur der Arbeit	2
2 Verwandte Arbeiten	4
2.1 Kollektive/kollaborative Performances und partizipative Kunst	4
2.2 Technische Grundlagen	6
2.2.1 Abbildungen von Kontrollflüssen	6
2.2.2 Partizipative Systeme	8
2.3 Verwandte Tools	9
2.3.1 TouchDesigner	9
2.3.2 Cables.gl	10
2.4 Abgrenzung zu verwandten Arbeiten	11
3 Konzeption	12
3.1 Use Cases	12
3.2 Anforderungen	14
3.3 Architektur	14
3.4 Mockups	18
4 Implementation	21
4.1 Architektur	21
4.1.1 Technologien und Komponenten	21
4.1.2 Aufbau	22
4.2 User Interface	24
4.2.1 Interaktion mit Peers	24
4.2.2 Nutzeranleitung	26
4.3 Deployment	27
4.4 Entwurf eines Demo-Workflows	27

5	Evaluation	29
5.1	Evaluationsdesign	29
5.2	Ergebnisse	31
6	Fazit und Ausblick	33
6.1	Beantwortung der Forschungsfrage	33
6.2	Kritische Reflexion	33
6.3	Ausblick	34
	Literatur	37
	Anhang	40

Tabellenverzeichnis

1	Funktionale Anforderungen	45
2	Nicht-funktionale Anforderungen	46

Abbildungsverzeichnis

2.1	Demonstrator der Installation „Speakers, More Speakers!!!“, Forgette et al., 2022	5
3.1	Konzipiertes System Architecture Diagram	15
3.2	Konzipiertes Sequenzdiagramm	16
3.3	Konzipiertes Klassendiagramm	17
3.4	Mockup der Startseite	19
3.5	Mockup des Editors	19
4.1	Das Interface des Editors	25
4.2	Beispiel für die Bidirektionalität der Software: Ein Canvas wird vom Host (Bildschirm oben) an ein anderes Gerät (unten rechts) gesendet.	26
4.3	Der erstellte Demo-Workflow auf Basis des Use-Cases „Partizpatives Kunstwerk“	27
1	Die Oberfläche von <i>TouchDesigner</i> , dargestellt wird das Demo-Projekt	40
2	Die Oberfläche von <i>cables</i> , dargestellt wird ein Projekt aus der Plattform	40
3	Die verfügbaren Nodes im Editor	41
4	Beispiel für ein Peer-Layout: Ein Klavier-Interface	42
5	Ausschnitt aus dem Manual	42
6	Das aktualisierte Klassendiagramm des implementierten Systems	43
7	Das aktualisierte Sequenzdiagramm des implementierten Systems	44

1 Einleitung

Für die Erstellung und Aufführung digitaler audiovisueller Kunstwerke stehen heutzutage vielfältige Softwarewerkzeuge zur Verfügung. Geläufig sind hier Digital Audio Workstations (DAWs) wie Ableton Live und grafische Programmierumgebungen wie Max/MSP für die Musikproduktion, sowie Game Engines wie Unity oder Unreal Engine oder Tools wie *TouchDesigner* für die Erstellung visueller Inhalte auf Basis von Audio. Viele der genannten Tools wie Max/MSP, die Unreal Engine oder *TouchDesigner* bieten dazu visuelle Programmier-Interfaces an, die es Nutzer:innen erlauben, komplexe Abläufe durch das Verbinden von modularen Bausteinen zu erstellen.

Eine weitere Form der Erstellung von audiovisuellen Performances hat sich in den letzten Jahren etabliert: das Konzept des Live-Codings. Hierbei schreiben Performer während der Aufführung Code, der in Echtzeit Klang- und Bildinhalte generiert oder verändert. Bekannte Umgebungen hierfür sind Sonic Pi, Strudel oder Hydra. Letztere beide sind webbasiert und ermöglichen das Nutzen neuartiger Web-APIs wie Sensor- und MIDI-Zugriff direkt im Browser.

Parallel dazu öffnen sich durch techniksoziologische Entwicklung wie die breite Adoption von Smartphones, sozialer Medien und Livestreaming neue Räume für kollaborative und partizipative Performances. Dies sind Auftritte oder Vorstellungen von Künstler:innen, bei welchen das Publikum oder andere Performer:innen aktiv in die Aufführung eingebunden werden.

Projekte wie das Stanford Mobile Phone Orchestra (MoPhO) (Oh et al., 2010; Wang et al., 2008) demonstrierten bereits 2008, dass Smartphones nicht nur Kommunikationsgeräte, sondern auch ernstzunehmende Instrumente innerhalb komplexer Klangensembles sein können. Neuere Arbeiten wie „Speakers, More Speakers!!!“ (Forgette et al., 2022) zeigen zudem, wie sich Smartphone-Sensorik, WebRTC und verteilte Interaktion zu neuartigen audiovisuellen Erfahrungen verweben lassen, inklusive experimenteller Setups wie dezentraler stochastischer Musik oder kollaborativ erzeugten Gemälden (Kanach und Nelson, 2024).

Trotz dieser Entwicklungen fehlt ein Werkzeug, das Live-Coding-Ansätze und kollaborative Performances insbesondere in einer gemeinsamen, zugänglichen Oberfläche vereint.

1.1 Forschungsfrage

Die Forschungsfrage, die dieses Projekt leiten soll, lautet:

Wie kann eine leicht zugängliche Oberfläche gestaltet werden, die es Künstler:innen und Performer:innen ermöglicht, audiovisuelle Inhalte in Echtzeit auf Basis von z.B. Sensordaten von mehreren Endgeräten zu orchestrieren?

1.2 Zielsetzung

Ziel dieser Arbeit ist die Entwicklung eines neuartigen Tools zur Nutzung in (Live-)Performances, mit welchem mithilfe des Publikums und dessen Endgeräten ein audiovisuelles Erlebnis gestalten kann.

Die Herausforderung stellt sich durch die Gestaltung eines solchen Tools: Wie könnte gleichzeitig ein Werkzeug geschaffen werden, das wie auch andere Werkzeuge einen freien Gestaltungsraum bietet, aber gleichzeitig zugänglich genug, um für Nutzer:innen mit wenig oder keinen Programmierkenntnissen nutzbar zu sein?

Im Rahmen dieser Arbeit wurde hierfür ein neuartiges System entwickelt. Zentraler Bestandteil ist ein Editor, in welchem diverse Interaktionen der Teilnehmer als Parameter verwendet werden können, um Visualisierungen zu erstellen und Klänge zu erzeugen, und auch von Nicht-Programmierer:innen genutzt werden kann.

Die Software ist, wie auch bei Live-Coding-Umgebungen oder vergleichbaren Tools, welche im weiteren Verlauf vorgestellt werden, so weit abstrahiert, dass der Nutzungsraum innerhalb der Domäne frei gestaltet werden kann, um kreative und innovative Ideen zu ermöglichen. Konkret wurde also eine visuelle Programmieroberfläche angestrebt, die es ermöglicht, Kontroll- und Datenflüsse zu erstellen. Damit sollen sowohl komplexe Strukturen zum Mapping von Teilnehmer-Daten erstellbar sein, als auch durch gute Usability und eine intuitive Interaktion die Einstiegshürde für neue Nutzer:innen niedrig gehalten werden.

Die Implementierung dieses Systems soll eine mögliche Antwort auf die gestellte Forschungsfrage liefern.

1.3 Struktur der Arbeit

Der Zeitraum für die Bearbeitung betrug 4 Monate, von Mitte Dezember 2025 bis Mitte April 2026. Die Arbeit wurde vor dem Projektstart in folgende Phasen gegliedert:

- **Vertiefende Anforderungsanalyse und Exploration bestehender Systeme** (ca. 2 Wochen)
 - **Literaturrecherche:** Analyse bestehender Literatur und verwandter Arbeiten im Bereich der Echtzeit-Orchestrierung audiovisueller Inhalte, insbesondere die Themen *Einbeziehung von Sensordaten* und *Partizipative und kollaborative Performances*.
 - **Systemanforderungen definieren:** Ausgehend von der Forschungsfrage und den Zielen des Projekts wurden Use Cases verfasst und daraus die funktionalen und nicht-funktionalen Anforderungen an das System abgeleitet.
- **Konzeption** (ca. 2 Wochen)
 - **System Architecture Diagram:** Erstellung von groben Diagrammen, die die Architektur des Systems skizzieren.
 - **Mockups/Wireframes:** Erstellung von Mockups der Benutzeroberfläche, um die Interaktion und das Design zu visualisieren.
 - **ADRs:** Vorläufige Dokumentation von Architektur- und Softwareentscheidungen in Form von Architecture Decision Records (ADRs), um die Überlegungen und Entscheidungen während der Konzeption zu konkretisieren und festzuhalten.
- **Implementierung** (ca. 5 Wochen)
 - **Architektur:** Aufbau der WebRTC-Kommunikation, Sensorstreaming, Peer-Management und QR-basiertem Pairing und Umsetzung erster Klang- und Visualisierungsbausteine sowie Mapping-Mechanismen.
 - **User Interface:** Entwicklung eines prototypischen Editors, der Sensordaten einbindet und mit welchem sich erste Performances erstellen lassen.
- **Evaluation und Iteration** (ca. 3 Wochen): Durchführung und Auswertung von Testsessions zu Bedienbarkeit, Verständlichkeit und Performance-Tauglichkeit und Fertigstellung eines stabilen Demonstrators für eine Beispielperformance.
- **Verschriftlichung** (ca. 2 Wochen): Verfassung der schriftlichen Ausarbeitung und weiterer Artefakte (Bilddokumentation, Abschlussvideo, etc.).
- **Puffer** (ca. 2 Wochen).

2 Verwandte Arbeiten

In diesem Kapitel werden verwandte wissenschaftliche Arbeiten sowie technische Produkte und Softwarelösungen vorgestellt, die sich in bestimmten Gesichtspunkten wie der Abbildung von Kontrollflüssen, der Generierung von audiovisuellen Inhalten oder der Nutzung von Sensordaten für Performances überschneiden.

Dabei wird auf verschiedene Teilaspekte der geplanten Zielsetzung eingegangen und vergleichbare Tools vorgestellt, um die Einordnung der vorliegenden Arbeit in den bestehenden Kontext zu ermöglichen und die Neuartigkeit sowie die Relevanz der Lösung herauszuarbeiten.

Aufgeteilt wird nach wissenschaftlichen Arbeiten, die sich zunächst mit der Thematik der kollektiven/kollaborativen Performances und partizipativen Kunst beschäftigen, und Arbeiten, welche sich nach technischen Aspekten mit der Abbildung von Kontrollflüssen und der Erstellung von audiovisuellen Inhalten beschäftigen. Es werden zudem etablierte Tools vorgestellt, die sich in derselben Domäne der Arbeit aufhalten.

2.1 Kollektive/kollaborative Performances und partizipative Kunst

In diesem Abschnitt wird sich auf die digitale Umsetzung von kollaborativen Performances fokussiert und Beispiele von realen Umsetzungen ähnlicher Konzepte vorgestellt.

Ein bekannter Vertreter solcher Performances ist das in der Einleitung bereits erwähnte Stanford Mobile Phone Orchestra (MoPhO) (Oh et al., 2010; Wang et al., 2008). Entwickelt wurde dort eine Plattform, welche für die zu der Zeit verbreiteten Nokia N95 und später Apple iPhones entwickelt wurde und diese so in Auftritte integrierte. Applikationen wurden gebaut, die mittels unterschiedlicher Interaktionen und speziellen Interfaces bestimmte Instrumente imitieren, welche in einem Konzert-Setting an Nutzer:innen basierend auf ihrer Position im Raum verteilt wurden und so ein Orchester imitiert wurde. Auch wenn es ein Ziel des Projektes war, wurde es im letzten Stand nicht geschafft, Nicht-Programmierer:innen die Möglichkeit zu geben, eigene Instrumente zu erstellen.

? Why



Abbildung 2.1: Demonstrator der Installation „Speakers, More Speakers!!!“, Forgette et al., 2022

Ebenfalls in der Einleitung wurde die Arbeit „Speakers, More Speakers!!!“ von Forgette et al. besprochen (Forgette et al., 2022).

Dabei handelt es sich um eine Installation, welche mehrere Lautsprecher und einen zentralen Bildschirm umfasst, sowie ein dazugehöriges webbasiertes System, welches Entwickler:innen mithilfe von Programmierschnittstellen die Möglichkeit bietet, interaktive audiovisuelle Erlebnisse zu gestalten, die von den Teilnehmer:innen über ihre Smartphones gesteuert werden können.

Genutzt wurden hierbei sowohl WebRTC für die Kommunikation zwischen den Geräten, als auch moderne Web-APIs, um Kamera, Mikrofon, Touch-Interaktionen und Accelerometer der Smartphones als Datenbasis für ihre Kunstinstallationen zu nutzen. Ein Bild aus der zitierten Arbeit ist in Grafik 2.1 zu sehen.

Xambo et al. beschäftigten sich während der CoViD-Pandemie mit der Integration von Teilnehmer:innen in eine live-gestreamten telematische Musikperformance (Xambó und Goudarzi, 2022). In Ihrer Arbeit stellen sie *personic* vor, ein System, in welchem Nutzer:innen per Link einer Performance beitreten können. Die Geodaten- und Bewegungssensoren ihrer Endgeräte wurden zur Erzeugung von Ambient-Musik und einer Visualisierung in Form einer 2D-Karte, auf der die Teilnehmer:innen als Blasen dargestellt werden, genutzt.

Eine weitere interessante Arbeit in diesem Kontext ist die von Lin et al., welche die Entwicklung eines Systems namens „CATtalk“ beschreibt (Lin et al., 2022). Es umfasst eine Plattform, mit welcher sich IoT-Geräte in einer Low-Code-Umgebung verbinden lassen. Ein in der Arbeit genanntes Anwendungsbeispiel war die Umsetzung einer Tanzchoreographie, in der die Bewegungen der Tänzer:innen an zwei geografisch

getrennten Orten in die Bewegung einer Kamera am jeweils anderen Ort übersetzt wurden, und gleichzeitig die Bewegungen der Tänzer:innen in Musik transformiert wurden.

Es gibt zahlreiche weitere Beispiele für ähnliche Projekte (z.B. Weitzner et al., 2013). Bei den Genannten handelt es sich um die mit der größten Überlappung zu den Projektzielen.

2.2 Technische Grundlagen

Im Folgenden sollen technische Aspekte der Umsetzung eines Systems besprochen werden, welches der formulierten Zielsetzung entspricht.

2.2.1 Abbildungen von Kontrollflüssen

Für eine technische Umsetzung eines in der Zielsetzung formulierten Systems steht zunächst die Frage nach der Abbildung und Editierung von Kontrollflüssen im Vordergrund. Zu dieser Thematik gibt es eine Vielzahl von Arbeiten und Softwarelösungen in verschiedenen Domänen mit der Bestrebung, die Erstellung von komplexen Abläufen einem breiten Publikum zugänglich zu machen, ohne dass tiefgehende Programmierkenntnisse erforderlich sind.

Ein prominentes Beispiel für den Einsatz innerhalb einer anderen Domäne, nämlich der Automatisierung von Geschäftsprozessen, ist die BPMN (Business Process Model and Notation)-Spezifikation (Chinosi und Trombetta, 2012), die eine standardisierte grafische Notation für die Modellierung und Automatisierung von Geschäftsprozessen bietet.

Auch eine Vielzahl von No-Code- und Low-Code-Plattformen, wie zum Beispiel Node-RED („Low-code programming for event-driven applications : Node-RED“, 2026) oder n8n (n8n GmbH, 2026), bieten visuelle Schnittstellen zur Erstellung von Abläufen durch das Verbinden von modularen Bausteinen, was eine ähnliche Zielsetzung verfolgt.

Was alle diese Tools eint, ist das Ziel von „End-User Development“ (EUD), in welchem Nutzer:innen, die ihre Domäne meist besser kennen als ein:e Entwickler:in, ein Werkzeug an die Hand gegeben wird, ohne professionelle Softwareentwickler:innen Lösungen für ihre Domäne zu schaffen.

Solche Tools, die mit visuellen Darstellungen von Abläufen arbeiten, werden auch als „Visual Programming Languages“ (VPLs) bezeichnet (vgl. Kuhail et al., 2021, p. 1-2).

In der Literatur wurde keine Arbeit gefunden, die sich mit der speziellen Domäne dieser Arbeit befasst.

Aus der Arbeit, die VPLs in anderen Domänen thematisieren, lassen sich jedoch interessante Erkenntnisse ableiten, die für die Gestaltung des geplanten Systems relevant sein können.

In der Arbeit von Kuhail et al., 2021 wird eine systematische Bewertung von verschiedenen VPL-Tools durchgeführt. Kategorisiert werden diese als „Form-based“, „Diagram-based“, „Block-based“ und „Icon-based“, je nach der Art der visuellen Repräsentation.

So werden bei „Block-based“ Tools einzelne Bausteine ineinander gesteckt, um Abläufe zu erstellen, während bei „Diagram-based“ Tools die Abläufe durch das Verbinden von Knoten in einem Graphen dargestellt werden (dieses Konzept wird in dieser Arbeit auch als „node-basierter“ Ansatz bezeichnet).

Ein Ergebnis dieser Arbeit ist die Erkenntnis, dass die meisten VPLs entweder blockbasiert oder node-basiert sind, wobei die blockbasierte Darstellung tendenziell in Lernumgebungen für Einsteiger, während die node-basierte Darstellung in professionelleren Domänen (z.B. Machine Learning, IoT) genutzt wird.

Auch wird festgestellt, dass die Evaluierungen dieser VPLs mehrheitlich empirisch ihren erfolgreichen Einsatz mit Endnutzer:innen demonstrieren konnten, sowie eine höhere Usability im Vergleich zu textbasierten Alternativen aufweisen konnten.

Dabei wird jedoch die Gestaltung der Tools ohne Nutzung der Basis von Usability-Guidelines von den Autoren kritisiert wird.

Eine weitere systematische Bewertung, speziell zu No-/Low-Code-Plattformen in der Domäne des maschinellen Lernens, wurde von Li und Wu, 2022 durchgeführt. Ihre Ergebnisse zeigen, dass Nutzer:innen die meisten Werkzeuge der klassischen Entwicklung bevorzugten.

Sie seien einfacher zu erlernen und effizienter zu nutzen, erhöhen die Produktivität und erleichtern standardisiertes und kollaboratives Arbeiten in Gruppen.

Die Arbeit listet außerdem genutzte Designprinzipien auf, die von den untersuchten Tools genutzt wurden. Dazu zählen:

- „Low threshold, high ceiling, and wide walls“ für einen einfachen Einstieg für Beginner und gleichzeitiger Möglichkeit zur Erstellung komplexer Projekte.
- Eine Informationsarchitektur, die mit dem mentalen Modell eines potentiellen Anfängers übereinstimmt.
- Unterstützung für schnelles Prototyping, Iteration und Feedback.

Es lässt sich ableiten, dass für das Erreichen der Zielsetzung dieser Arbeit die node-basierte Darstellung und Editierung von Abläufen aufgrund der Spezifik und potentiellen Komplexität der Domäne passend wäre. Es müssen Möglichkeiten geschaffen

werden, um die Einstiegshürde für Anfänger so weit wie möglich zu reduzieren, indem Interaktionen an das mentale Modell dieser Nutzer:innen angepasst werden, aber dennoch ein breites Nutzungsspektrum ermöglicht wird. Einfach zugängliches Feedback und direkte Ausgaben von Zwischenergebnissen können dabei helfen, die Lernkurve zu reduzieren und schnell zu prototypisieren.

2.2.2 Partizipative Systeme

Die vorgestellten Arbeiten in Abschnitt 2.1 haben verschiedene Ansätze gezeigt, wie man kollaborative oder partizipative Performances gestalten oder orchestrieren kann. In diesem Abschnitt sollen die technischen Aspekte dieser Arbeiten genauer betrachtet werden.

MoPhO setzte hierbei auf die Verteilung der Teilnehmer:innen im physischen Raum. Die Geräusche wurden direkt vom Telefon der Teilnehmer:innen abgespielt. Dieser Ansatz hat dadurch Limitationen; z.B. müssen alle Teilnehmer:innen physisch anwesend sein, auch mussten Lösungsansätze gefunden werden, um die Lautsprecher der Telefone zu amplifizieren.

Ein Ansatz, der versuchte, Performances auf technische Weise zu orchestrieren, ist die von Hödl et al., 2020. Sie nutzten eine eigens implementierte App für ihre Performance, welche mithilfe von Ultraschall und der Mikrofone der Telefone dem Publikum einer Live-Musikperformance die Teilhabe ermöglichte.

Schnell und Matuszewski entwickelten zudem eigens für diesen Zweck eine Bibliothek namens Soundworks (siehe Matuszewski et al., 2019; Schnell und Robaszkiewicz, 2015).

Diese nutzt ein Client/Server-Modell und ist in JavaScript implementiert, und nutzt mithilfe von „socket.io“ WebSockets für die Kommunikation zwischen Server und Clients.

Mit der Bibliothek lassen sich für Programmierer:innen kollaborative audiovisuelle Performances erstellen, die Bibliothek übernimmt hierbei die Übertragung von Client-Daten wie Sensor- und Toucheingaben an den Server, welcher die Daten zu einer Performance zusammenführt. Denselben Ansatz mittels Client/Server-Modell und der Nutzung von socket.io für die Kommunikation verfolgte auch die Arbeit von Xambó und Goudarzi, 2022.

Die Arbeit von Forgette et al., 2022 nutzte WebRTC für die Kommunikation zwischen den Geräten.

WebRTC (Web Real-Time Communication) ist eine Sammlung von APIs, die die Audio-, Video- und Datenkommunikation zwischen Browsern ermöglichen (Sredojev et al., 2015).

WebRTC besitzt mehrere Vorteile für den Einsatz in dieser Domäne: Zunächst ermöglicht es eine Kommunikation in nahezu Echtzeit, da die Verbindung direkt zwischen Teilnehmern (oder „Peers“) hergestellt wird, ohne dass die Daten über einen zentralen Server geleitet werden müssen.

Zum Anderen ist es auf die Übertragung von Audio- und Videodaten optimiert, was es ideal für die Übertragung von Sensordaten und die Synchronisation von audiovisuellen Inhalten macht.

2.3 Verwandte Tools

Um konkretere Vergleiche zu ziehen, werden im Folgenden zwei Tools vorgestellt, die sich mit der Erstellung von audiovisuellen Inhalten beschäftigen und dabei teilweise auch Aspekte der Abbildung von Kontrollflüssen oder die Integration von oder Synchronisierung mit Daten aus den Interaktionen des Publikums oder anderer Performer beinhalten.

Die Auswahl der Tools basiert auf ihrer Relevanz für die Zielsetzung der Arbeit, ihrer Bekanntheit in der Community und ihrer Funktionalität im Bereich der audiovisuellen Orchestrierung.

2.3.1 TouchDesigner

TouchDesigner ist ein Tool zur Generierung von Visualisierungen in Echtzeit, mit einem Fokus auf der Erstellung von Visuals auf Basis von Audio-Input (vgl. „TouchDesigner“, 2026). Basis ist ein node-basiertes, visuelles Programmierinterface, mit welchem sich eine große Vielfalt von Operationen durchführen und Schnittstellen anbinden lassen. *TouchDesigner* stellt eine Vielzahl spezialisierter Operatoren (OPs) bereit, die jeweils bestimmte Aufgaben innerhalb der Datenverarbeitung übernehmen. Diese sind in unterschiedliche Kategorien eingeteilt, dazu zählen TOPs (Texture Operators) zur Bildverarbeitung, CHOPs (Channel Operators) zur Verarbeitung zeitabhängiger Daten wie Audio oder Sensorwerte, SOPs (Surface Operators) für Geometrie sowie DATs (Data Operators) zur Handhabung von Tabellen, Text oder Skripten.

Die Benutzeroberfläche ist in mehrere Bereiche gegliedert. Im Mittelpunkt steht der „Network Editor“, in dem die einzelnen Operatoren miteinander verknüpft werden. Ergänzt wird dieser durch spezielle, Operator-spezifische Ansichten wie den Parameter-Editor, den Viewer zur Vorschau von Zwischenergebnissen sowie zusätzliche Fenster für Skripting oder Performance-Analyse. Eine Besonderheit von *TouchDesigner* ist, dass Nutzer:innen während der Laufzeit Parameter verändern können und sofort

visuelles Feedback erhalten, was insbesondere für experimentelle Arbeitsprozesse von Vorteil ist.

In Grafik 1 ist die Oberfläche von *TouchDesigner* zu sehen, dargestellt wird das Demo-Projekt, welches mit der Software mitgeliefert wird. Sichtbar ist ein Workflow, welches ein Bild von Jellybeans mithilfe von einem Noise-Operator verzerrt und auf einen Würfel als Textur gemappt wird. Gut sichtbar sind hier die Vorschau-Boxen der einzelnen Operationen, für Nutzer:innen ist sofort ersichtlich, welchen Effekt die Operationen auf das Endergebnis haben.

Tatsächlich bietet *TouchDesigner* auch Operationen für das Nutzen von Networking, sogar WebRTC im Speziellen. Dies sind allerdings nur technische Schnittstellen; um diese wirklich nutzen zu können, werden Programmierkenntnisse benötigt.

2.3.2 Cables.gl

cables ist eine webbasierte Plattform zur Erstellung von Echtzeit-Visualisierungen, die ebenfalls auf einem node-basierten, visuellen Programmieransatz basiert (vgl. „cables“, 2026). Ähnlich wie bei *TouchDesigner* werden einzelne Funktionseinheiten als Knoten dargestellt und über Verbindungen zu Verarbeitungsketten kombiniert.

Die Plattform stellt eine Vielzahl von „Ops“ (Operatoren) bereit, welche unterschiedliche Aufgaben übernehmen, darunter insbesondere die Verarbeitung von WebGL-Grafik, Audio-Daten sowie Eingaben aus verschiedenen Quellen. Ein Fokus liegt dabei auf der direkten Integration von Webtechnologien, wodurch Inhalte unmittelbar im Browser ausgeführt und veröffentlicht werden können.

Die Grafik 2 zeigt die Benutzeroberfläche mit einem geöffneten Workflow. Im Gegensatz zu *TouchDesigner* sind die Knoten minimalistischer gestaltet und der Kontrollfluss wird von oben nach unten dargestellt. An den Knoten werden alle Eingabe- und Ausgabeparameter des Knotens angezeigt. Diese sind typisiert, so haben beispielsweise fast alle Knoten einen „Trigger“-Parameter, welcher dessen Ausführung auslöst und in den meisten Fällen von Knoten zu Knoten weitergegeben werden kann, um so einen Kontrollfluss zu erstellen. Die Datentypen werden dabei als visuelle Hilfe für Nutzer:innen mit Farben signalisiert.

Zudem lässt sich auch eine Kontrolloberfläche innerhalb des Outputs selbst erstellen, was neue Interaktionsmöglichkeiten mit dem generierten Inhalt ermöglicht. Wie bei *TouchDesigner* können Parameter während der Laufzeit angepasst werden.

Teil von der Webseite *cables.gl* ist zudem eine Community-Plattform, auf der Projekte direkt online geteilt und ausgeführt werden können. Darüber hinaus besteht

die Möglichkeit, eigene JavaScript-Funktionen zu integrieren, um weitere Nutzungsmöglichkeiten zu schaffen, die nicht mit der visuellen Programmierlogik abgebildet sind.

Möglichkeiten zum Networking sind in *cables* ebenfalls vorhanden. Es existieren Operatoren für WebSockets und HTTP-Requests; diese Integrationen sind allerdings nicht direkt angedacht für partizipative Performances, sondern eher z.B. für die Anbindung von APIs.

A

2.4 Abgrenzung zu verwandten Arbeiten

Wie in den vorherigen Abschnitten beschrieben, gibt es eine Vielzahl von Arbeiten und Tools, die in Teilaspekten mit der vorliegenden Arbeit verwandt sind.

Die Neuartigkeit dieser Arbeit besteht in der Kombination von Software zur Erstellung dynamischer audiovisueller Inhalte, der Integration von Daten der Teilnehmer:innen und der Abbildung von Kontrollflüssen in einer einzigen, zugänglichen Softwarelösung. Ein solches System existiert nach der Recherche nicht.

3 Konzeption

Die Konzeptionsphase diente der Konkretisierung der Zielsetzung und der Formulierung von Anforderungen an die geplante Softwarelösung.

Die Recherche zu verwandten Tools diente dabei als Inspiration und half, die Möglichkeiten und Grenzen bestehender Lösungen zu verstehen, um darauf aufbauend eine eigene Lösung zu entwickeln, die der Zielsetzung des Projekts entspricht.

Es wurde mit einem webbasierten Ansatz geplant, da dadurch die Zugänglichkeit und die Möglichkeit zur kollaborativen Nutzung über verschiedene Endgeräte mittels einer Code-Basis hinweg ermöglicht wird, ohne dass eine spezielle Software installiert werden muss, was in vielen der in Kapitel 2 vorgestellten Tools nicht oder höchstens eingeschränkt gegeben ist.

Limitiert würde die Software dabei teils durch den webbasierten Ansatz, da nur Funktionen und Möglichkeiten genutzt werden können, die durch Web-APIs bereitgestellt werden. Da es jedoch zum Zeitpunkt der Durchführung dieses Projekts eine Vielzahl von Web-APIs gibt, die Zugriff auf Sensoren, Kamera, Mikrofon, Vibrationsmotor und andere Funktionen von Smartphones ermöglichen, ist die Bandbreite der möglichen Anwendungen dennoch sehr groß.

3.1 Use Cases

Das Nutzungsspektrum für die geplante Softwarelösung ist breit gefächert, da die geplante Anwendung für kreative Nutzung angedacht sind.

Drei mögliche Nutzungsszenarien wurden formuliert, teils inspiriert aus bestehendem Werk (siehe: 2.1). Diese sollen das zu erzielende breite Nutzungsspektrum konkretisieren.

Dies sind die drei formulierten Use Cases:

1. **Musik-Performance:** Ein:e Performer:in (z.B. Musiker:in) möchte bei einem Live-Auftritt mit seinem:ihrem Publikum musizieren. Dafür ruft die Person die Software auf und erstellt einen neuen Raum.
Nutzer:innen können über einen alphanumerischen Beitrittscode oder per QR-Code dem Raum beitreten. Der oder die Performer:in sieht die Teilnehmer:innen,

die sich verbunden haben.

Der oder die Performer:in stellt ein, dass er oder sie das Accelorometer der Teilnehmer:innen benutzen möchte. Die Daten werden zu der Person in die Anwendung gestreamt.

Genutzt werden die einkommenden Daten durch den:die Performer:in zum Beispiel:

- als Parameter für Sounddesign, zum Beispiel, indem die Verbindungen als Obertöne genutzt werden und die Bewegungsgeschwindigkeit die Lautstärke bestimmt.
- als Parameter für interaktive Visuals, die zusammen mit der Audio der Performance zur Musikvisualisierung genutzt werden.
- als Trigger für Samples, zum Beispiel, indem die Bewegung über einem bestimmten Schwellenwert ein Schlagzeug-Sample auslöst.

Zusätzlich stellt der:die Performer:in ein, dass er oder sie die Kamera der Teilnehmer:innen benutzen möchte. Der Kamerafeed von zufälligen Teilnehmer:innen, die die Kameraberechtigung erteilt haben, werden teils als Audience Cameras für Konzert-Visuals übernommen oder in diese integriert.

Der oder die Performer:in nutzt die App weiterhin, um an die Teilnehmer:innen Daten zu schicken. So stellt er oder sie zum Beispiel ein, dass auf Basis der Eingangsaudio, oder an bestimmten Stellen in der Performance, oder Steuerbar über ein MIDI-Pedal oder ein anderes Hardware-Eingabegerät, das Smartphone der Teilnehmer:innen in dem Raum vibriert.

2. **Kunst-Installation:** Ein:e Künstler:in stellt ein partizipatives Kunstprojekt in einen öffentlichen Raum. Es besteht aus einem großen Bildschirm. Ein QR-Code/Raum-Key ist dort sichtbar angebracht, mit dem sich Passant:innen verbinden können. Der oder die Künstler:in hat mittels der App einen Workflow eingerichtet, dass, wenn sich neue Nutzer:innen verbindet, diese einen Bereich auf der Leinwand zugewiesen kriegen, und auf ihrem Smartphone eine Zeichenfläche dargestellt kriegt. Diese kann nun frei gestalten, ohne bestehende Inhalte auf der Leinwand übermalen zu können. Es entsteht ein kollaboratives Gemälde, welches von zufälligen Passant:innen gestaltet wird.

3. **Livestreaming:** Ein:e Streamer:in möchte ein partizipatives Event mit seinen:ihren Zuschauer:innen gemeinsam gestalten. Gestreamt wird die Audio- und Videoausgabe der App, sowie ein QR-Code zum Beitreten.

A

Der oder die Streamer:in sieht neu eingehende Verbindungen und weist diesen eine Rolle zu, z.B. ein Instrument zum Spielen, indem er oder sie für die Verbindung einstellt, welche GUI zurückspielt der:dem entsprechenden Nutzer:in angezeigt wird. In den Einstellungen des Nodes wählt der oder die Streamer:in aus den vorkonfigurierten Layouts ein Klavier-Layout aus, eventuell könnte er oder sie aber auch ein eigenes mit Code schreiben oder in einem Editor erstellen. Zudem richtet er oder sie ein, dass, wenn der oder die Nutzer:in mit dem GUI interagiert, eine entsprechende Aktion stattfindet, z.B. der entsprechende Klavierton abspielt wird.

So entsteht eine etwas chaotische, theoretisch über die ganze Welt verteilte Band.

Ersichtlich ist, dass abseits von der Nutzung in Live-Performances auch vorprogrammierte Abläufe denkbar sind.

3.2 Anforderungen

Ausgehend von den formulierten Use Cases wurden die funktionalen und nicht-funktionalen Anforderungen an das System abgeleitet. Nicht alle der Anforderungen sind für alle Use Cases relevant. Solche, die als Eckpfeiler der grundsätzlichen Funktionalität des Systems angesehen werden können, wurden als „Must“ klassifiziert.

In der Tabelle 1 sind die funktionalen Anforderungen aufgelistet, in Tabelle 2 die nicht-funktionalen Anforderungen.

3.3 Architektur

Basierend auf den Grundideen der Zielsetzung und den formulierten Anforderungen wurde eine grobe Systemarchitektur skizziert. Gestaltet wurden drei Diagramme: ein System Architecture Diagram (3.1), ein Sequenzdiagramm (3.2) und ein Klassendiagramm (3.3).

Die Diagramme wurden mit dem Tool *Mermaid* erstellt.

System Architecture Diagram Das System Architecture Diagram zeigt grob die Hauptkomponenten des Systems. Zentral steht das „Web-based Orchestration Tool“, was sowohl die Schnittstelle für den Host (Performer/Artist) als auch für die Peers darstellt. Diese könnten theoretisch zwei verschiedene Anwendungen sein, eine monolithische Umsetzung ist jedoch von Vorteil, da sich so Code (wie Klassen und

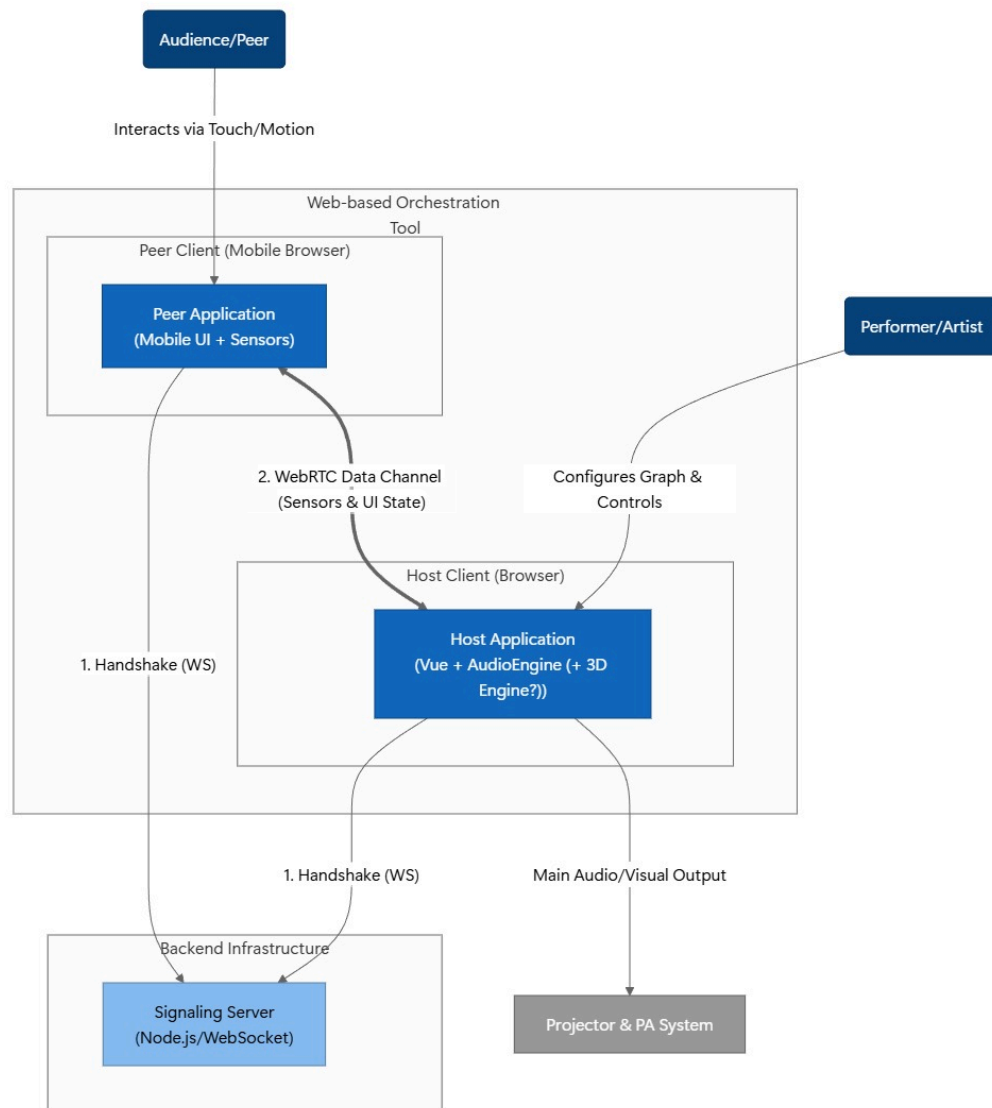


Abbildung 3.1: Konzipiertes System Architecture Diagram

Interfaces) und Funktionalität (wie Ablaufprotokolle) teilen lassen, was die Entwicklung erleichtert. Die Verbindung dieser beiden Teile findet direkt über WebRTC statt.

Wie aus der Grafik ersichtlich ist zudem eine Backend-Infrastruktur skizziert. Diese

ist für die Session Description im WebRTC-Verbindungsaufbau nötig, sowie um die Verwaltung von Sessions und Räumen zu übernehmen. Es kann hier eine eigene Lösung implementiert oder auf bestehende Lösungen zurückgegriffen werden, wie sie von WebRTC-Frameworks wie PeerJS („Simple peer-to-peer with PeerJS | PeerJS“, 2026) bereitgestellt werden.

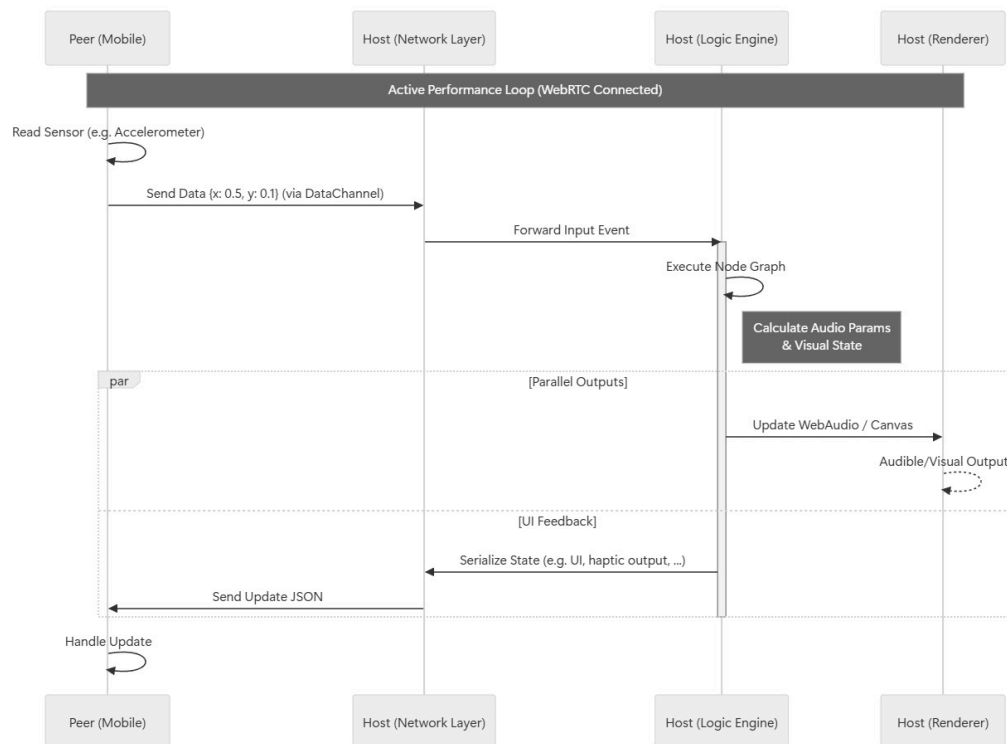


Abbildung 3.2: Konzipiertes Sequenzdiagramm

Sequenzdiagramm Das Sequenzdiagramm zeigt einen beispielhaften Ablauf vom Datenfluss zwischen Peer und Host. Nachdem eine WebRTC-Verbindung aufgebaut wurde, werden wiederholt Daten vom Peer zum Host gesendet. Dies enthält, wie in der Grafik genannt, zum Beispiel die Sensorwerte, aber auch Interaktionen mit der Benutzeroberfläche und Audio- und Videodaten über die entsprechenden Media Streams, je nachdem, welche Berechtigungen der oder die Nutzer:in erteilt hat.

Der Host übernimmt in dieser Konzeption eine mit einer *Selective Forwarding Unit* vergleichbaren Rolle. Diese Architektur ist gängig bei WebRTC-basierten Anwendun-

gen. Hier werden Peers nicht in einem Mesh verbunden, sondern laufen über einen zentralen Dienst, der die Streams empfängt und verarbeitet, um sie dann an die entsprechenden Peers zurückzuschicken.

In der Grafik ist der Host in drei Teile aufgeteilt: das Network Layer, die Logik-Engine und der Renderer. Das Network Layer empfängt die Daten und leitet sie an die Logik-Engine weiter, welche die Daten verarbeitet und entsprechend der vom Performer definierten Abläufe und Mappings in der Software entscheidet, welche Aktionen ausgeführt werden sollen.

Diese Aktionen können zum Beispiel sein, dass ein bestimmter Sound abgespielt wird, oder dass eine Visualisierung auf der Leinwand verändert wird, oder dass ein Vibrationsmuster auf den Endgeräten der Teilnehmer:innen ausgelöst wird. In letzterem Fall sendet die Logik die entsprechenden Befehle zurück an die Peers, welche diese dann über das Network Layer empfangen und ausführen.

A

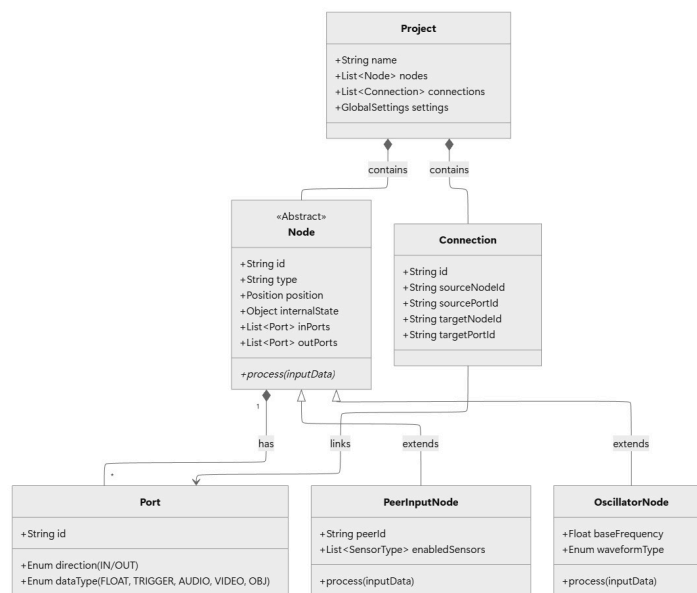


Abbildung 3.3: Konzipiertes Klassendiagramm

Klassendiagramm Das Klassendiagramm ist relativ simpel, es stellt die Klassenstruktur dar, um einen Node-based Editor zu implementieren. Hauptklasse ist das „Project“, welches die Datenstruktur des Projekts enthält, also die Informationen über die Nodes, Verbindungen und allgemeine Einstellungen.

Die allgemeinen Einstellungen des Projekts sind flexibel und könnten zum Beispiel die Informationen über den Verbindungscode enthalten. Die abstrakte „Node“-Klasse stellt die Basis für alle Arten von Nodes dar. Dargestellt in der Grafik sind zwei exemplarische Subklassen, der „PeerInputNode“ und der „OscillatorNode“ - es werden jedoch viele weitere Node-Typen implementiert werden.

In diesem Beispiel wäre der „PeerInputNode“ ein Node, welcher die Daten eines verbundenen Peers repräsentiert, also zum Beispiel die Sensorwerte oder die Interaktionen mit der Benutzeroberfläche des Peers, während der „OscillatorNode“ ein Node wäre, welcher einen Oszillator repräsentiert, also einen Klanggenerator, welcher zum Beispiel über die Web Audio API implementiert werden könnte.

Nodes besitzen eine Position innerhalb des Editors, sowie eine Liste von Ein- und Ausgängen, welche als „Ports“ betitelt sind.

Wie auch ein Kontrollfluss als Konzept aus der Programmierung stammt und in Form eines Graphen dargestellt wird, ist auch die Darstellung von Datentypen nicht für der oder die Nutzer:in abstrahierbar.

Als Beispiel: Würde ein oder eine Nutzer:in die eingehenden Audiodaten vom Mikrophon eines Peers verarbeiten wollen, sind für diese Daten auch nur Transformationsmöglichkeiten relevant, die für diesen Datentypen auch Sinn ergeben.

Sowohl *TouchDesigner* 2.3.1 als auch *cables* 2.3.2 lösen dieses Problem, indem die Knoten des Graphen selber typisiert sind und typisierte Ein- und Ausgänge haben und nur mit kompatiblen Ausgängen verbunden werden können.

A

3.4 Mockups

Um einen konkreten Eindruck von der geplanten Benutzeroberfläche zu vermitteln und eine Idee zur konkreten Umsetzung der Peer-Orchestrierung zu schaffen, wurden Mockups mit Figma erstellt. Diese sind in Abbildung 3.4 und 3.5 zu sehen. Es wurden lediglich Mockups für einen Desktop-Viewport erstellt, Responsive Design ist für den Editor nicht Teil der Anforderungen.

Die Hauptseite in Abbildung 3.4 zeigt eine einfache Startseite und muss nicht detailliert besprochen werden. Sie enthält lediglich einen Button zum Erstellen eines neuen Raums, sowie die Möglichkeit, einem bestehenden Raum beizutreten.

Der Editor in Abbildung 3.5 zeigt die grundlegende Struktur der Benutzeroberfläche, dessen Design-Entscheidungen im Folgenden näher erläutert werden. Dazu sei gesagt, dass dies nur eine Skizze der finalen Oberfläche ist und dass Abweichungen in beispielsweise der Positionierung von Elementen und Funktionen oder Ästhetik in dem finalen Layout zu erwarten waren.



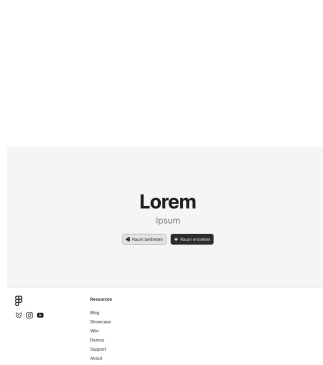


Abbildung 3.4: Mockup der Startseite

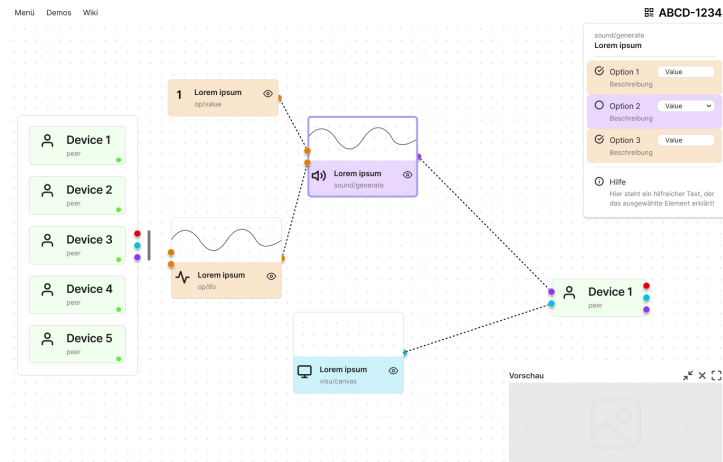


Abbildung 3.5: Mockup des Editors

Über die Menüleiste am oberen Rand können allgemeine Aktionen aufgerufen werden, außerdem werden hier der Join-Code und der QR-Code angezeigt, um neue Teilnehmer:innen zum Beitreten einzuladen. Hier befindet sich zudem die Referenz zu einer Anleitung.

Der Hauptbereich der Oberfläche besteht aus dem Node-Editor, sowie einem Panel auf der linken Bildschirmseite, welches die verbundenen Peers anzeigt. Wie im Teil 3.3 beschrieben, werden die verbundenen Peers als Nodes zur Verfügung gestellt. Gedacht ist, dass die Peers aus dem Panel in den Node-Editor gezogen werden können, um so die Daten der Peers in die Logik einzubinden. Zudem bietet das Peer-Panel selbst Ausgabeports, was die Option bieten soll, die Daten aller Peers zu verarbeiten.

Beispielhaft dargestellt ist ein einfacher Workflow, welcher Audio- und Videodaten an einen bestimmten Peer senden würde, mit einer entsprechenden beispielhaften Operation, wie sie im Graph erstellt werden würde. Gebaut werden diese aus Operatoren (Nodes), welche über Verbindungen (Edges) anhand deren Ein- und Ausgänge (Ports) miteinander verbunden werden können.

Im Fokus steht die Klanggenerator-Operation (der „*sound/generate*-Node“), dessen Einstellungen durch Klick auf diesen geöffnet werden. In dem Einstellungs-Panel befinden sich Operator-spezifische Einstellungen und Hilfestellungen, die diese erklären. Um eine der Einstellungen mit einer Variable aus dem Graphen zu verbinden, kann der oder die Nutzer:in die Checkbox für den Parameter aktivieren, wodurch ein entsprechender Ausgang (oder „Port“) auf dem Node erscheint, welcher mit einem Port eines anderen Nodes verbunden werden kann, womit die ausgegebenen Daten dieses Ports als Parameter genutzt werden.

Wie im vorangegangenen Abschnitt beschrieben, sind Ports wie auch in vergleichbaren Softwarelösungen typisiert. Die Darstellung der Typen erfolgt in diesen mittels Farben. Dieses Konzept wurde auch für das Mockup der Oberfläche übernommen. So wäre im erstellten Mockup zum Beispiel die Ausgabe des Canvas-Node hellblau, und entsprechende Outputs ließen sich nur mit Inputs desselben Typs verbinden.

Ebenfalls das Konzept der Vorschau für einzelne Nodes ist in *TouchDesigner* zu finden. Im Mockup befindet sich dafür exemplarisch eine Vorschau für die Canvas- und Audio-Nodes. Die Ausgabe des Canvas-Nodes wäre in diesem Fall der Echtzeit-Stand der Visualisierung, welche auf der Leinwand dargestellt wird, während die Vorschau des Audio-Nodes die aktuelle Wellenform des generierten Klangs anzeigt. In *TouchDesigner* geht dieser visuelle Output für einige Knoten-Typen weiter: so lässt sich mit einigen Knoten auch interagieren oder teils Einstellungen innerhalb dieser Visualisierung vornehmen. Diese Interaktionsmöglichkeit wird in dieser Software nicht implementiert, da Einstellungen für die Nodes über das Einstellungs-Panel als zentralen Ort vorgenommen werden sollen, wäre aber eine denkbare Interaktionsmöglichkeit.

In der unteren rechten Ecke befindet sich letztendlich der Ausgabebereich, welcher die Visualisierung auf der Leinwand darstellt. Diese soll sich in einem neuen Tab öffnen lassen, um so die Möglichkeit zu bieten, die Visualisierung frei auf dem Bildschirm des Hosts zu positionieren.

4 Implementation

In diesem Kapitel wird die Implementierung des Systems beschrieben, das zur Beantwortung der Forschungsfrage entwickelt wurde. Es werden die Architektur, die verwendeten Technologien und die wichtigsten Komponenten des Systems erläutert. Sowohl der Programmcode als auch Texte auf der Oberfläche inklusive Anleitung wurden auf Englisch verfasst, um die Zugänglichkeit für ein internationales Publikum zu gewährleisten. Eine Internationalisierung der Oberfläche ist möglich und auch sinnvoll, wurde aber im Rahmen dieser Arbeit nicht umgesetzt.

In der Entwicklung wurden KI-gestützte Tools (GitHub Copilot, IntelliJ Junie) genutzt, um die Entwicklung durch Scaffolding, sowohl der Architektur als auch der Benutzeroberfläche zu erleichtern. Für GitHub Copilot wurden die Modelle Claude 4.5 und 4.6 von Anthropic sowie Codex 4.3 von OpenAI genutzt.

4.1 Architektur

Wie im Abschnitt 3.3 beschrieben, basiert die Architektur des Systems auf einen Peer-to-peer-Ansatz, mit einem Host als zentralem Koordinator. Im Folgenden werden sowohl die Entscheidungen, welche Technologien eingesetzt wurden, sowie die wichtigsten Komponenten und deren Interaktionen beschrieben.

4.1.1 Technologien und Komponenten

Bei dem Programm handelt es sich um eine Single-Page-Webanwendung, welche mit Vue.js und TypeScript entwickelt wurde.

Da nur für den Verbindungsaufbau ein Server notwendig ist, welcher aber von der PeerJS-Bibliothek bereitgestellt wird, wurde auf die Entwicklung eines eigenen Servers verzichtet.

Genauere Informationen, wieso sich für die jeweiligen Technologien entschieden wurde, wurden in den ADRs dokumentiert, welche im Repository des Werks zu finden sind (siehe Gerwing, 2026).

4.1.2 Aufbau

WebRTC-Verbindungsmanagement Für die WebRTC-Verbindungen von Peer und Host wird die PeerJS-Bibliothek eingesetzt, dessen Signalisierungsserver genutzt wird, um die Verbindungsherstellung zu ermöglichen.

Wenn der Host einen neuen Raum erstellt, wird ein achtziffriger Code generiert und der PeerJS-Bibliothek als Peer-ID übergeben. Dieser Code kann entweder manuell von den Teilnehmer:innen eingegeben oder durch Scannen eines QR-Codes, welcher den Code enthält, genutzt werden, um dem Raum beizutreten.

Das Verbindungsmanagement läuft dabei abgetrennt von der Graph-Logik. Das Verbindungsmanagement wird von einem Pinia-Store „sessionStore“ verwaltet, der, wenn Daten von Peers empfangen werden, diese in den „peerStore“ weiterleitet, welcher der Graph-Logik als Schnittstelle dient, um auf die Daten der Peers zuzugreifen. Der „peerStore“ agiert hier auch gleichzeitig als Puffer, da die eingehenden Verbindungsdaten je nach Netzwerk zeitlich verteilt eintreffen und nicht in synchronem Timing sind mit der Ausführung des Graphen.

Graph-Logik und -Ausführung Wie auch die Peers und die Session wird zur Speicherung der Kanten und Knoten des Graphen ein Pinia-Store „graphStore“ genutzt, welcher vom Benutzer über die Oberfläche bearbeitet wird.

Beim Ausführen des Graphen wird mithilfe der „requestAnimationFrame“-Funktion ein Loop gestartet, welcher in jedem Frame den Graphen durchläuft und die Logik der Nodes ausführt. Diese Ausführung übernimmt eine „GraphExecutor“-Klasse. Diese sortiert den Graphen anhand der Verbindungen topologisch, um die Reihenfolge der Ausführung zu ermitteln.

Anschließend werden die „process“-Methoden der einzelnen Nodes nacheinander aufgerufen, welche die Logik der Nodes implementieren. Diese Funktion beschreibt dabei die Daten der Output-Ports, die anschließend in die verbundenen Input-Ports der nächsten Nodes kopiert werden. Jeder Teilbaum und somit *jeder* Knoten des Graphen wird abgearbeitet.

Die Logik des Graphen ist nicht Event-basiert, d.h. es gibt keine Trigger, welche die Ausführung der Nodes starten. Dieser Ansatz hat den Vorteil, dass es für neue Nutzer:innen einfacher ist, den Kontrollfluss nachzuvollziehen: Wenn „Start“ gedrückt wird startet die Ausführung; es braucht keine spezielle Definition des Kontrollflusses wie z.B. bei *cables* (siehe 2.3.2).

Dass der Visitor alle Knoten des Graphen abarbeitet, hat Einfluss darauf, wie Knoten, die den Kontrollfluss steuern sollen („If“-Node, „Loop“-Node) sich verhalten. Der „If“-Node zum Beispiel gibt abhängig von der Bedingung die eingehenden Daten

entweder an den „True“- oder den „False“-Output-Port weiter. Da die Nodes die im jeweils anderen Zweig liegen trotzdem ausgeführt werden, werden an sie leere Daten weitergegeben, welche von diesen Nodes nicht weiterverarbeitet werden.

Der Loop-Node löst das gleiche Problem, indem er einen internen Zähler hat, mit welchem bei jedem Frame das nächste Element der Liste der Daten, welche er als Input erhält, weitergegeben wird, bis die Liste abgearbeitet ist. Dies verhindert auch potenzielle Probleme mit großen Datenmengen, da nicht alle Daten auf einmal weitergegeben werden, sondern über mehrere Frames verteilt.

Diesen Ansatz zu wählen, war eine Entscheidung, welche sich erst während der Implementierung herausgestellt hat und einen einfacheren Umgang mit der grundsätzlichen Logik der Software ermöglichen soll, da es näher am mentalen Modell neuer Nutzer:innen liegt. Ob dies der Fall ist, kann in der Evaluation ermittelt werden.

Operatoren und Nodes Zum Zeitpunkt der Verschriftlichung dieses Dokuments wurden viele verschiedene Node-Typen implementiert, welche in Grafik 3 ersichtlich ist. Dargestellt wird dort die Übersicht über die verfügbaren Operationen, wie sie auch in der Software zu finden ist.

Die Operationen sind in folgende Kategorien eingeteilt:

- **Peer Nodes:** Repräsentieren die verbundenen Peers und machen die Daten der Peers, wie z.B. Sensordaten oder andere Interaktionen, innerhalb des Editors verfügbar. Die Peer-Nodes werden nicht in der Node-Library angezeigt, sondern sind dynamisch basierend auf den verbundenen Peers generiert.
- **Video Nodes:** Operationen, welche zur Erstellung von Video-Outputs genutzt werden können.
- **Audio Nodes:** Operationen, welche zur Erstellung von Audio-Outputs genutzt werden können.
- **Processing Nodes:** Erlauben Verarbeitung von Daten, z.B. mathematische Operationen, Logik oder das Verwenden von Variablen.
- **Control Flow Nodes:** Erlauben die Steuerung von Abläufen (Bedingungen und Schleifen)
- **Output Nodes:** Operationen, welche die Ausgabe von Audio oder Video ermöglichen, z.B. durch das Öffnen eines neuen Tabs mit einem Canvas-Output oder die Ausgabe von Klängen. Auch das Zurückgeben der Daten an alle Peers ist über einen Output Node möglich.

- **Utility Nodes:** Hilfsoperationen, welche zur Entwicklung oder Fehlersuche genutzt werden können, z.B. zum Logging von Daten oder zum Erstellen von Notizen/Kommentaren.

Erwähnenswert ist die Integration von Common Expression Language (Google, 2026) in einigen Nodes, um die Erstellung von komplexeren Logiken in einer simplifizierten Sprache zu ermöglichen.

4.2 User Interface

Für die Umsetzung der Benutzeroberfläche wurde die Komponentenbibliothek DaisyUI verwendet, um simple, häufig benötigte UI-Elemente wie Buttons, Modals u.ä. nutzen zu können, ohne diese selber implementieren und gestalten zu müssen.

Grundsätzlich ist der Aufbau der Oberfläche in der Struktur gleich zum Mockup, welches in der Konzeption erstellt wurde (siehe 3.4). Zwei Abweichungen zum Mockup wurden bei der Implementation vorgenommen: Zum einen hat das Peer Panel nicht wie angedacht Output-Ports, um alle Daten der Peers verfügbar zu machen, da das Panel an der linken Seite des Editors fixiert ist und im zoom- und scrollbaren Editor die Verbindungen mit anderen Nodes unübersichtlich wäre. Es kann stattdessen ein „All Peers Node“ aus dem Panel in die Oberfläche gezogen werden. Zum anderen wurde die kleine Änderung vorgenommen, den visuellen Output in der linken unteren Ecke anzuzeigen, damit Platz für einen Knopf zum Aufrufen der Knotenauswahl im rechten unteren Bereich des Editors zu schaffen, welcher in den Mockups gefehlt hat. Die Oberfläche des Editors ist in Abbildung 4.1 zu sehen.

4.2.1 Interaktion mit Peers

Wie in der Konzeption angedacht, wurde auch eine bidirektionale Kommunikation zwischen Host und Peers implementiert. So können nicht nur Daten von den Peers im Editor genutzt, sondern auch Daten zurück an die Peers gesendet werden. Im Editor äußert sich dies so, dass Peers auch über Eingangs-Ports verfügen. Im Bild 4.2 wird dies demonstriert.

Um weitere Interaktionsmöglichkeiten neben den Sensormessungen der Peers zu schaffen, wurde, wie in den Anforderungen festgehalten (Abschnitt 3.2), die Möglichkeit implementiert, dem Peer ein Layout anzuzeigen. Gelöst ist dies, indem der Host eine Reihe an vordefinierten Layouts für jeden Peer präsentiert bekommt; und diese entweder direkt im Peer-Panel oder in den Node-Einstellungen eines Peers ausgewählt

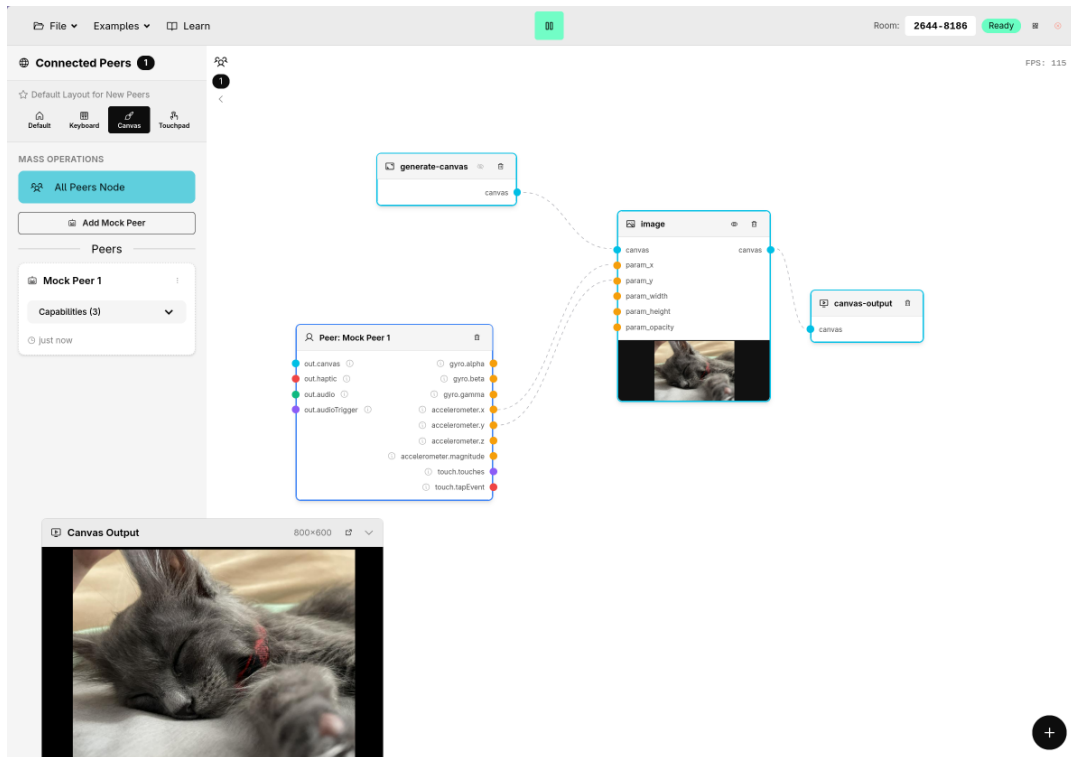


Abbildung 4.1: Das Interface des Editors

werden können. Es ist zudem möglich, ein Standard-Layout zu setzen, das neu verbundenen Peers automatisch angezeigt wird.

Je nach ausgewähltem Layout werden neue Daten-Ports auf dem Peer-Node erstellt, welche die Interaktionen der Peers repräsentieren. So hat der Host als Beispiel beim Auswählen des „Keyboard“-Layouts, welches dem Host ein Klavier-Interface darstellt (siehe Abbildung 4), einen neuen Output-Port „frequency“ für diesen Peer. Dieser kann direkt mit einem Tongenerierungs-Node verbunden werden; ein Peer kann somit Melodien spielen, die auf dem Host wiedergegeben werden.

Die Layouts wurden als Vue.js-Komponenten implementiert. Hier wurde auf Responsive Design geachtet, da die Nutzung auf allen möglichen Endgeräten funktionieren soll.

Es existierte die „Could“-Anforderung, eine Schnittstelle zu schaffen, über welche Nutzer:innen eigene Layouts erstellen und in die Software integrieren können, um die Möglichkeiten der Interaktion zu erweitern. Dies ist aber im Rahmen dieser Arbeit aus zeitlichen Gründen nicht umgesetzt worden, ist für eine zukünftige Weiterentwicklung

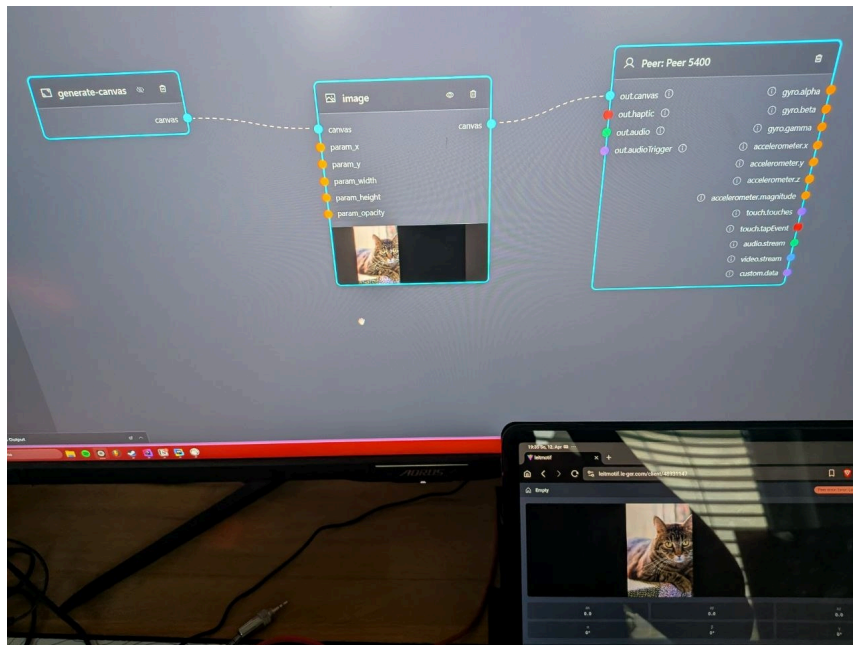


Abbildung 4.2: Beispiel für die Bidirektionalität der Software: Ein Canvas wird vom Host (Bildschirm oben) an ein anderes Gerät (unten rechts) gesendet.

jedoch angedacht (siehe 6.3).

4.2.2 Nutzeranleitung

Für die Nutzung der Software wurde eine Anleitung erstellt, welche in der Navigationsleiste der Software referenziert ist. Ein Ausschnitt der Anleitung ist in Abbildung 5 zu sehen.

Sie richtet sich sprachlich an Nutzer:innen, die keine oder nur wenig Erfahrung mit derartigen Tools haben, und soll die wichtigsten Funktionen und Konzepte der Software erklären, um den Einstieg zu erleichtern.

Es existieren zudem Abschnitte und Erklärungen, welche in feinerem Detail auf Aspekte der Software eingehen, wie zum Beispiel das Nutzen von CEL oder die Verarbeitungslogik von Canvas und Audio.

Teil des Manuals ist eine vollständige Referenz aller Nodes, inklusive Erklärungen zu den Funktionen und Parametern der Nodes. Um diese schnell zugänglich zu machen, existiert in dem Einstellungs Menü der Nodes ein Button, welcher die jeweilige Node-Referenz im Manual öffnet.

4.3 Deployment

Für die Software ist ein Dockerfile vorhanden, welches zum virtualisierten Deployment genutzt wird. Der Server, auf dem die Software zur Zeit der Fertigstellung der Arbeit läuft, wird vom Ersteller der Arbeit privat gemietet. Die Anwendung wurde vorerst dort deployt, um die Software für die Evaluation und Demonstration verfügbar zu machen.

4.4 Entwurf eines Demo-Workflows

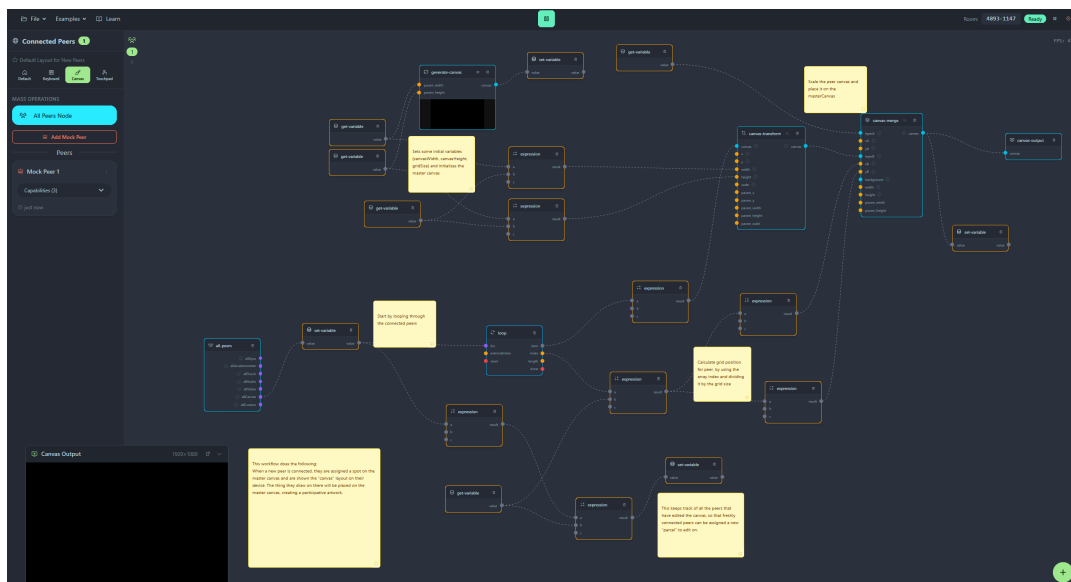


Abbildung 4.3: Der erstellte Demo-Workflow auf Basis des Use-Cases „Partizpatives Kunstwerk“

Teil der Implementierung war auch die Erstellung von mindestens einem Demo-Workflow. Dieser sollte für die Evaluation genutzt werden, um eine Aufgabe gestalten zu können, in welcher eine komplexere Struktur bearbeitet werden muss. Er als auch der die Demonstration dienen, um die Möglichkeiten der Software aufzuzeigen.

Erstellt wurde eine Umsetzung des zweiten Use-Cases, wie er im Abschnitt 3.1 beschrieben wurde. Der Grund für die Wahl dieses Use-Cases liegt darin, dass dieser

einen konkreten und leicht vermittelbaren Anwendungsfall darstellt.

Der Workflow wurde mit Hilfskommentaren ausgestattet, der die Funktionalität des Workflows allgemein und einzelne Abschnitte der Logik erklärt.

Der erstellte Workflow ist in Abbildung 4.3 zu sehen.

5 Evaluation

Folgend wird die Evaluation des entwickelten Systems und dessen Ergebnisse beschrieben, um die Forschungsfrage zu beantworten und die Effektivität der Lösung zu bewerten.

5.1 Evaluationsdesign

Da der Fokus auf der Zugänglichkeit der Benutzeroberfläche liegt, muss die Evaluation darauf ausgerichtet sein, die kognitive Belastung der Nutzer:innen zu erfassen und die Benutzbarkeit der Software selbst zu ermitteln.

In der „Cognitive Load Theory“ von Sweller, 2011 wird unterschieden zwischen dem intrinsischen Cognitive Load, der durch die Komplexität der Aufgabe selbst entsteht, und dem extrinsischen Cognitive Load, der durch die Gestaltung der Benutzeroberfläche verursacht wird. Problematisch kann hier der intrinsische Cognitive Load des Inhalts der Software sein, da gewisse Konzepte neuen Nutzer:innen möglicherweise unbekannt sind.

Ein Beispiel hierfür ist die Generierung von Klängen: Für ein:e Nutzer:in, welche:r noch nie mit der Erzeugung von Klängen durch Software oder Synthesizern in Berührung gekommen ist, kann die Nutzung der Software aufgrund dessen als zu komplex empfunden werden.

Ziel der Evaluation war es daher, den durch die Benutzeroberfläche verursachten Extraneous Cognitive Load qualitativ und unterstützend quantitativ möglichst genau zu erfassen.

Der Mixed-Methods-Ansatz kann damit aufzeigen, inwieweit die Software die Zielsetzung erfüllen kann.

Zur quantitativen Erfassung wurde das System Usability Scale (SUS) verwendet. Das System Usability Scale wurde gewählt, da es ein etabliertes und weit verbreitetes Instrument zur Messung der Usability von Systemen ist und eine gute Vergleichbarkeit mit anderen Systemen ermöglicht.

Ebenso wurde der Einsatz anderer Usability-Fragebögen erwägt. Die Arbeit von Assila et al., 2016 vergleicht verschiedene Usability-Fragebögen, darunter das SUS, aber auch andere geläufige Fragebögen wie das Software Usability Measurement Inventory

(SUMI).

Die Fragebögen wurden nach den dort abgefragten Kriterien kategorisiert, unter anderem „Satisfaction“, „Helpfulness“ etc.. Von den zwanzig verglichenen Fragebögen untersuchen lediglich drei das von den Autoren benannte „Ease of use / Usability“-Kriterium, zu welchen auch das SUS gehört.

Da dieses Kriterium die passendste Metrik für die Fragestellung ist, wurde das SUS als quantitatives Instrument gewählt.

Es wurde eine Studie mit acht Teilnehmer:innen durchgeführt, dessen Ablauf folgendermaßen gegliedert war:

1. Vorerhebung Erfassung von demografischen Daten und Abfrage von Vorkenntnissen mittels Rating-Skala von 0-10 (Programmierung, visuelle Tools, Audio/Visual-Systeme).

2. Einführung Eine kurze Einführung in Grundkonzepte des Editors sowie grundlegende Interaktionen.

3. Aufgabenbearbeitung Die Teilnehmenden bearbeiteten zwei Aufgaben unterschiedlicher Komplexität:

Task 1: Open Creation Erstellung eines einfachen eigenen Projekts, in welchem Sensor-Daten zur Soundgenerierung genutzt werden sollen.

Task 2: Modification Task Verständnisfragen und Anpassungsaufgabe eines komplexeren bestehenden Projekts.

Während der Aufgabenbearbeitung wurden die Teilnehmenden aufgefordert, ihre Gedanken zu verbalisieren.

5. Nachbefragung Nach Abschluss der Aufgaben wurde das SUS eingesetzt, um unterstützende quantitative Daten zu erhalten.

Zusätzlich wurden offene Fragen gestellt, um subjektive Eindrücke und Verbesserungsvorschläge zu erfassen, sowie Feedback zur Evaluation selbst, um die Methodik zu reflektieren.

6. Auswertung Die Studie wurde begleitet mit Bildschirm- und Audioaufzeichnungen der Teilnehmer:innen. Die Auswertung erfolgte durch das Kodieren der Probleme nach Kategorien (z.B. Navigationsprobleme, Missverständnisse), der Auswertung der SUS-Fragebogen und weiteren gestellten Fragen mit einer kontextualisierung durch die von den Teilnehmern angegebene Erfahrung.

5.2 Ergebnisse

Von den Teilnehmer:innen hatten sechs Vorkenntnisse in Programmierung. Ebenfalls sechs Teilnehmer:innen gaben an, Erfahrung mit Audio-/Visualizations-Software zu haben. Alle Teilnehmer:innen hatten keine oder geringe Erfahrung (<5) mit visuellen Programmier-Tools.

Die Bearbeitungsdauer der Aufgaben betrug etwa 17 Minuten im Minimum und etwa 38 Minuten im Maximum, mit einem Durchschnitt von etwa 28 Minuten.

In der Regel teilte sich die Bearbeitungszeit gleich auf die beiden Aufgaben auf.

Aus Bildschirmaufnahmen und Aufzeichnungen ließen sich folgende Probleme beim Bewältigen der Aufgabe identifizieren:

- Unklarheit über die Terminologie: Begriffe wie „Graph“, „Canvas“ und „Peer“ waren teilweise unklar, was zu Verwirrung führte.
- Schwierigkeiten bei der Navigation: Einige Teilnehmer:innen hatten Schwierigkeiten, bestimmte Funktionen zu finden, wie zum Beispiel den „Raum erstellen“-Button, der den QR-Code generiert.
- Unklarheit über Interaktion: Einige Teilnehmer:innen waren unsicher, was sie tun müssen, um Knoten und Peer-Knoten in der Oberfläche hinzuzufügen.

Diese Probleme traten dabei vor allem in der ersten Aufgabe auf.

Vor allem in Kombination mit den Schwierigkeiten in der Navigation sind zudem Verständnisprobleme zur Aufgabe und den Funktionalitäten der Software aufgetreten.

In drei Fällen gab es die gleiche Situation, bei der in der Knoten-Bibliothek vergeblich nach einem „Input“ für die Sensor-Daten gesucht wurde. Dieses Problem wurde dann selbstständig gelöst, als die Teilnehmer:innen das Peer-Panel entdeckt haben.

Positiv hervorheben lässt sich, dass die grundlegenden Interaktionen und das Prinzip des Datenflusses schnell verstanden wurden.

Alle Teilnehmer:innen waren zudem in der Lage, die Aufgaben selbstständig zu bewältigen, unabhängig von ihrer angegebenen Erfahrung in den Domänen.

Bei zwei der Nutzer:innen hat die Nutzung der Dokumentation die Lösung der Aufgaben ermöglicht.

Die Auswertung des SUS-Fragebogens gab einen Durchschnittswert von 68,125 Punkten.

Um einen Vergleich mit anderen Systemen zu ziehen: In der Arbeit von Bangor et al., 2009 wurde eine große Zahl an Systemen untersucht und deren SUS-Werte extrahiert. Im Durchschnitt lag der SUS-Wert für Web-Interfaces bei 68,2 Punkten und für Software, die mit dem Adjektiv „gut“ bewertet wurde, bei 71,4 Punkten, mit einer Standard-Abweichung von 11,6. In diese Kohorte lässt sich auch die Punktzahl, die für dieses System ermittelt wurde, einordnen.

Es konnte keine Korrelation zwischen der angegebenen Erfahrung der Teilnehmer:innen und der Bewertung der Usability festgestellt werden.

6 Fazit und Ausblick

Abschließend wird die Forschungsfrage beantwortet, die Zielsetzung reflektiert und ein Ausblick auf mögliche Erweiterungen gegeben.

6.1 Beantwortung der Forschungsfrage

Die Evaluationsergebnisse 5.2 zeigen, dass die im Rahmen dieser Arbeit erstellte Software die Zielsetzung erfüllt und somit eine zugängliche Lösung für das Erstellen von kollaborativen audiovisuellen Performances bietet.

Limitationen in dieser Aussage bestehen darin, dass die Evaluation mit einer kleinen Stichprobe von acht Teilnehmern durchgeführt wurde, was die Generalisierbarkeit der Ergebnisse einschränkt.

Weitere Forschung könnte sich auf die Durchführung einer größeren Studie mit einer vielfältigeren Teilnehmergruppe konzentrieren, um die Ergebnisse zu validieren und ein umfassenderes Verständnis der Usability und Zugänglichkeit der Software zu erlangen.

6.2 Kritische Reflexion

In der Zeitplanung wurde ein Puffer von zwei Wochen eingeplant. Dieser war auch notwendig, da vor allem die Implementierung mehr Zeit in Anspruch genommen hat als ursprünglich gedacht.

Die Implementation des Projektes lief chaotischer ab als erhofft. Die zuvor durchgeführte Konzeption hat sich als nicht weit genug gedacht herausgestellt, weshalb die tatsächliche Architektur in Teilen von der ursprünglichen Planung abweicht.

So gibt es zum Beispiel nicht wie ursprünglich angedacht eine zentrale „Projekt“-Klasse, die die Graph-Struktur und die Logik der Nodes verwaltet, sondern mehrere auf Pinia basierende „Stores“, die für die Verwaltung der Daten zuständig sind, sowie den „Graph Executor“, in welchem die Prozedur für die Ausführung der Logik implementiert ist. Ursache für die Diskrepanz besteht dabei nicht in der Schwäche der Konzeption an sich, sondern daran, dass bestimmte Patterns und Strukturen

sinnhafter sind bei der Arbeit in der Web- beziehungsweise der Frontend-Entwicklung mit Frameworks wie Vue.

Gemeint ist damit: Wie im Beispiel genannt, hätte die ursprünglich geplante „Projekt“-Klasse zwar in der Form implementiert werden können, der Weg über domänenbezogene Stores und die Trennung von Datenverwaltung und Logik ist aber näher an der gängigen Praxis in der Webentwicklung mit dem gewählten Tech-Stack. Die konkrete Planung, welche Technologien eingesetzt werden sollen, wurde erst nach der Konzeption abgeschlossen, was möglicherweise ein Fehler war.

Die Abbildungen 6 und 7 zeigen aktualisierte Diagramme, welche die tatsächliche Architektur des implementierten Systems widerspiegeln.

Ebenfalls hätte in der Konzeption konkretisiert werden müssen, welche Features nötig sind, um ein vollständiges visuelles Programmierinterface zu schaffen. ✓

Zur Umsetzung des Demo-Workflows musste zum Beispiel ein Variablen-System implementiert werden, was zusätzlich Zeit in Anspruch genommen hat.

Durch die fehlende Zeit gab es weniger Zeit für die Evaluation und somit war es nicht mehr möglich, auf Basis der Ergebnisse zu iterieren, bevor die Arbeit abgegeben werden musste.

Positiv anmerken lässt sich, dass genau an den richtigen Stellen Fremdlösungen genutzt wurden, die die Entwicklung deutlich vereinfacht haben oder Features ermöglicht haben, ohne, dass auf zu viele Fremdlösungen zurückgegriffen werden musste.

Die Nutzung von Agents und KI-gestützter Code-Generierung war hilfreich und zeitsparend, um komplexe Logiken zu implementieren, insbesondere im Bereich der WebRTC-Verbindungsverwaltung und der Peer-Layouts. Sie wurde in den Teilen problematisch, wo der Kontext zu groß wurde oder das Prompting die KI in eine unerwünschte Richtung führte.

Für die Evaluation sollten in einer Wiederholung der Studie andere Aufgaben gestellt werden, um die Ergebnisse zu validieren.

6.3 Ausblick

Wie in 6.2 bereits erwähnt, gab es aufgrund von Zeitmangel keine Möglichkeit, die Ergebnisse der Evaluation in die Entwicklung einfließen zu lassen. Bei den Tests sind mehrere Probleme aufgefallen, die durch kleinere UI-Verbesserungen behoben werden könnten, welche wären:

- Das Peer-Panel sollte standardmäßig ausgeklappt sein

- Der „Raum erstellen“-Knopf muss prominenter platziert werden, am besten als Teil des Peer-Panels
- Vergrößerung von wichtigen Elementen
- Anpassungen von Textformulierungen
- Bessere Feedback-Mechanismen bei Fehlern
- Vereinfachung von bestimmten Logiken aus dem 4.4, die mit CEL gelöst wurden (z.B. das Arbeiten mit Listen)

Außerdem bräuchte das System zur vollen Abdeckung aller Anforderungen noch einige Features und Bugfixes.

Unter anderem sind ab und zu Verbindungsprobleme aufgetreten. Des Weiteren gibt es aktuell Probleme bei dem Canvas-Output in neuem Tab.

Auch die Implementierung von eigenen Layouts ist aktuell nicht implementiert.

Es müssten zudem neue Node-Typen implementiert werden, was jedoch durch den modularen Aufbau gut möglich ist. Dazu zählen:

- Audio-Processing und Integration von MIDI-Schnittstellen
- Audio-Effekte (ist einfach implementierbar mit Tone.js)
- Video-Processing und Merging in Canvas
- Generierung von Visualisierung - P5 oder Three.js
- KI-Modelle

Wie in 4 erwähnt wurde keine Internationalisierung umgesetzt. In den Evaluationen haben die Begriffe „Peer“ und „Canvas“ teilweise zu Verwirrung geführt, weshalb eine Internationalisierung mit angepasster Terminologie eine sinnvolle Ergänzung wäre, um die Zugänglichkeit zu erhöhen.

Eine Verbesserung der Dokumentation mit weiteren Beispielen und Tutorials ist ebenfalls sinnvoll, auch um weitere Demonstratoren für das System zu schaffen.

Eine weitere denkbare Weiterentwicklung wäre die wie bei *cables* 2.3.2 existierende Möglichkeit, eine Plattform aufzubauen, um Projekte mit der Öffentlichkeit zu teilen und zu remixen.

Eine solche Plattform-basierte Erweiterung könnte die Reichweite und den Einfluss der Software deutlich erhöhen.

Denkbar wäre eine wirtschaftliche Nutzung einer solchen Plattform, zum Beispiel durch die Einführung von Premium-Features oder einem Abonnement-Modell, um die Nachhaltigkeit des Projekts zu gewährleisten.

Auf längere Sicht soll das Projekt jedoch Quelloffen und frei zu nutzen bleiben, welcher der künstlerischen und kollaborativen Natur des Projekts entspricht.

Literatur

- Assila, A., Oliveira, K., & Ezzedine, H. (2016). Standardized Usability Questionnaires: Features and Quality Focus. *Computer Science and Information Technology*. Verfügbar 1. April 2026 unter <https://www.semanticscholar.org/paper/Standardized-Usability-Questionnaires%3A-Features-and-Assila-Oliveira/99dc3571e17aa41bca454605b5386b330b2409a6>
[TLDR] This paper presents the result of a detailed analysis of all items being evaluated in each questionnaire to indicate those that can identify users' perceptions about specific usability problems and confronts each questionnaire item with usability criteria proposed by quality standards and classical quality ergonomic criteria.
- Bangor, A., Kortum, P., & Miller, J. T. (2009). Determining what individual SUS scores mean: adding an adjective rating scale. *Journal of Usability Studies archive*. Verfügbar 17. April 2026 unter <https://www.semanticscholar.org/paper/Determining-what-individual-SUS-scores-mean%3A-adding-Bangor-Kortum/3399f83ff6149dc65b52600f52ed372be5a6aa86>
[TLDR] The addition of the adjective rating scale to the SUS may help practitioners interpret individual SUS scores and aid in explaining the results to non-human factors professionals.
- cables. (2026). Verfügbar 12. April 2026 unter <https://cables.gl/>
- Chinosi, M., & Trombetta, A. (2012). BPMN: An introduction to the standard. *Computer Standards & Interfaces*, 34(1), 124–134. <https://doi.org/10.1016/j.csi.2011.06.002>
- Forgette, A., Manaris, B., Gillikin, M., & Ramdsen, S. (2022). "Speakers, More Speakers!!! – Developing Interactive, Distributed, Smartphone-Based, Immersive Experiences for Music and Art". *Proceedings of the International Symposium on Electronic Art*. Verfügbar 14. Januar 2026 unter <https://par.nsf.gov/biblio/10374099-speakers-more-speakers-developing-interactive-distributed-smartphone-based-immersive-experiences-music-art>
- Gerwing, L. (2026, April). lea-ger/leitmotif [original-date: 2025-11-11T17:12:19Z]. Verfügbar 9. April 2026 unter <https://github.com/lea-ger/leitmotif>
- Google. (2026). Common Expression Language (CEL). Verfügbar 12. April 2026 unter <https://cel.dev/overview/cel-overview/>

- Hödl, O., Bartmann, C., Kayali, F., Löw, C., & Purgathofer, P. (2020). Large-scale audience participation in live music using smartphones. *Journal of New Music Research*, 49(2), 192–207. <https://doi.org/10.1080/09298215.2020.1722181>
- Kanach, S., & Nelson, P. (Hrsg.). (2024, Oktober). *Meta-Xenakis: New Perspectives on Iannis Xenakis's Life, Work, and Legacies* (1. Aufl.). Open Book Publishers. <https://doi.org/10.11647/OBP.0390>
- Kuhail, M. A., Farooq, S., Hammad, R., & Bahja, M. (2021). Characterizing Visual Programming Approaches for End-User Developers: A Systematic Review. *IEEE Access*, 9, 14181–14202. <https://doi.org/10.1109/access.2021.3051043>
- Li, L., & Wu, Z. (2022). How Can No/Low Code Platforms Help End-Users Develop ML Applications? - A Systematic Review. In *Lecture Notes in Computer Science* (S. 338–356). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-21707-4_25
- Lin, Y.-B., Luo, H., & Liao, C.-C. (2022). CATtalk: An IoT-Based Interactive Art Development Platform. *IEEE Access*, 10, 127754–127769. <https://doi.org/10.1109/ACCESS.2022.3227093>
- Low-code programming for event-driven applications : Node-RED. (2026). Verfügbar 14. April 2026 unter <https://nodered.org/>
- Matuszewski, B., Schnell, N., & Bevilacqua, F. (2019). Interaction Topologies in Mobile-Based Situated Networked Music Systems [eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1109/Wireless_Communications_and_Mobile_Computing.2019.9142490]. *Wireless Communications and Mobile Computing*, 2019(1), 9142490. <https://doi.org/10.1155/2019/9142490>
- n8n GmbH. (2026). AI Workflow Automation Platform - n8n. Verfügbar 14. April 2026 unter <https://n8n.io/>
- Oh, J., Herrera, J., Bryan, N., Dahl, L., & Wang, G. (2010). Evolving the Mobile Phone Orchestra.
- Schnell, N., & Robaszkiewicz, S. (2015). Soundworks – A playground for artists and developers to create collaborative mobile web performances. *Proceedings of the Web Audio Conference (WAC'15)*. Verfügbar 12. April 2026 unter <https://hal.science/hal-01580797>
- Simple peer-to-peer with PeerJS | PeerJS. (2026). Verfügbar 9. April 2026 unter <https://peerjs.com/>
- Sredojev, B., Samardzija, D., & Posarac, D. (2015). WebRTC technology overview and signaling solution design and implementation. *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, 1006–1009. <https://doi.org/10.1109/MIPRO.2015.7160422>
- Sweller, J. (2011, Januar). Cognitive Load Theory. In *Psychology of Learning and Motivation* (S. 37–76, Bd. 55). Academic Press. <https://doi.org/10.1016/B978-0-12-387691-1.00002-8>

- TouchDesigner. (2026). Verfügbar 12. April 2026 unter <https://derivative.ca/UserGuide/TouchDesigner>
- Wang, G., Essl, G., & Henri, P. (2008). Do Mobile Phones Dream of Electric Orchestras.
- Weitzner, N., Freeman, J., Chen, Y.-L., & Garrett, S. (2013). massMobile: towards a flexible framework for large-scale participatory collaborations in live performances. *Organised Sound*, 18(1), 30–42. <https://doi.org/10.1017/S1355771812000222>
- Xambó, A., & Goudarzi, V. (2022). The Mobile Audience as a Digital Musical Persona in Telematic Performance. *NIME 2022*. <https://doi.org/10.21428/92fbeb44.706b549e>

Anhang

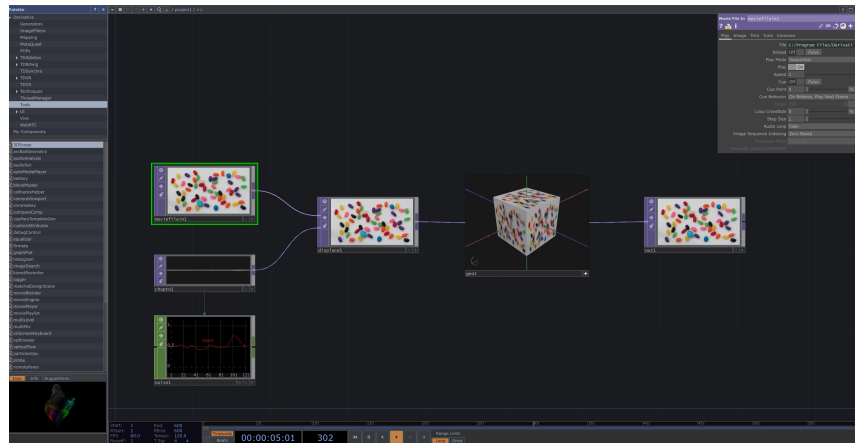


Figure 1: Die Oberfläche von *TouchDesigner*, dargestellt wird das Demo-Projekt

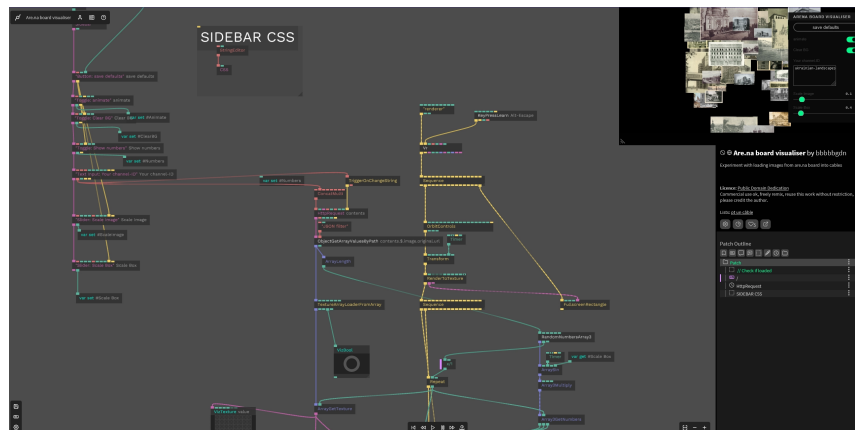


Figure 2: Die Oberfläche von *cables*, dargestellt wird ein Projekt aus der Plattform

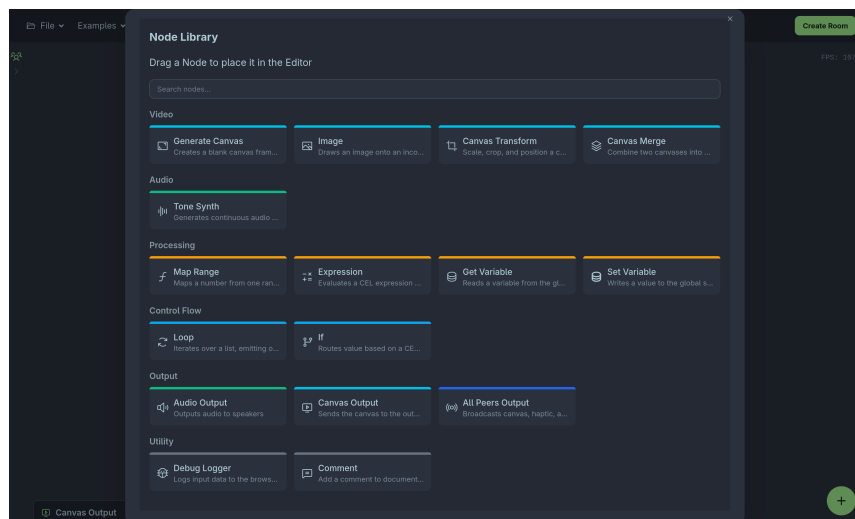


Figure 3: Die verfügbaren Nodes im Editor

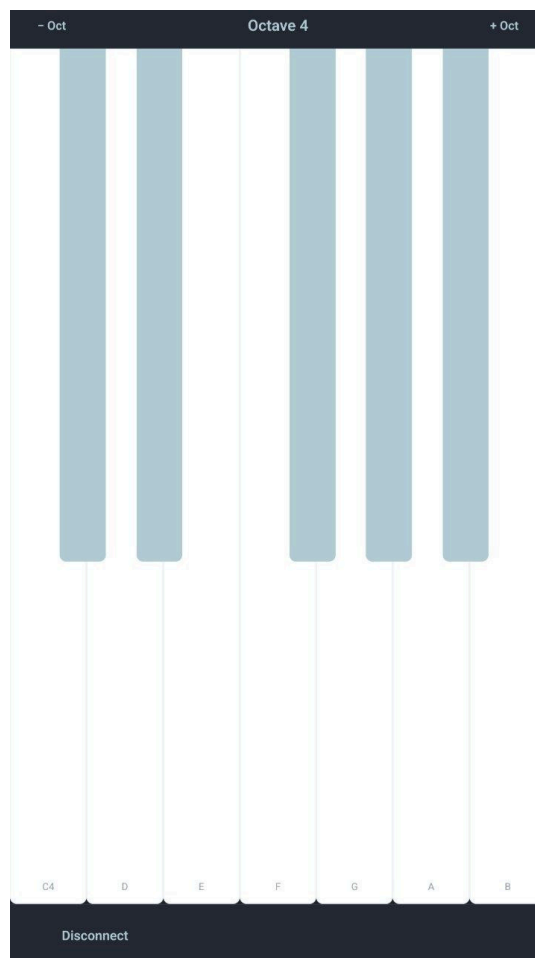


Figure 4: Beispiel für ein Peer-Layout: Ein Klavier-Interface

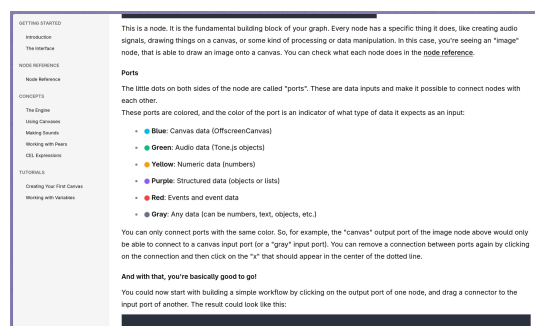


Figure 5: Ausschnitt aus dem Manual

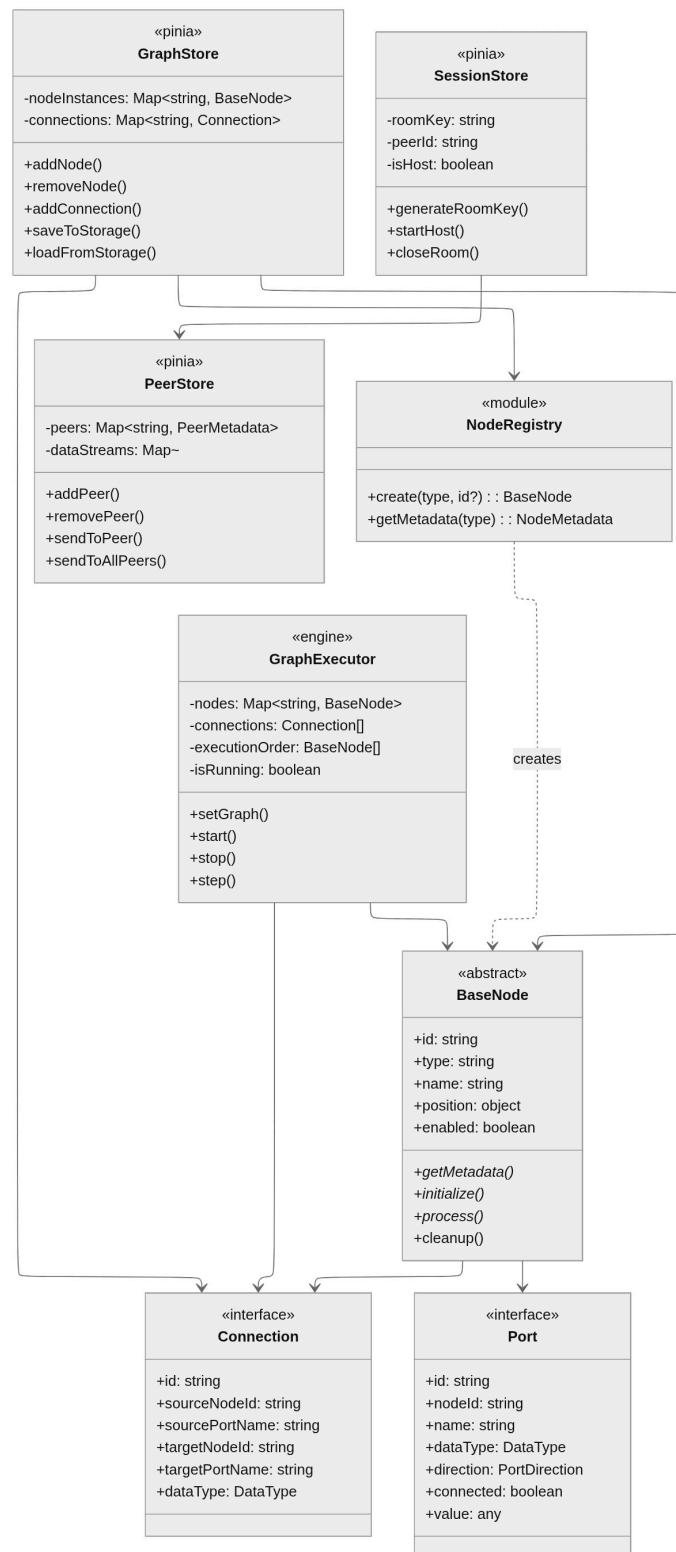


Figure 6: Das aktualisierte Klassendiagramm des implementierten Systems

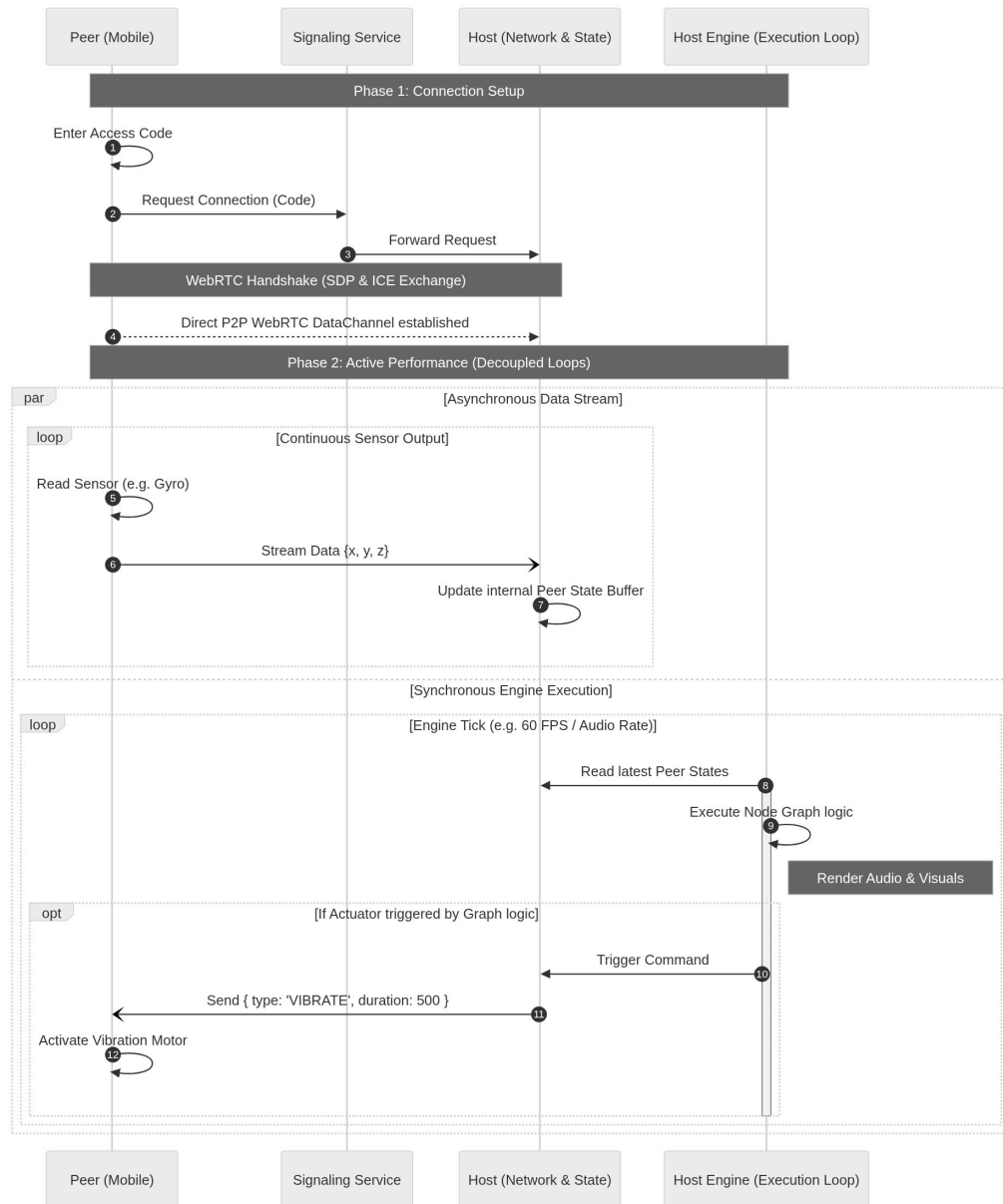


Figure 7: Das aktualisierte Sequenzdiagramm des implementierten Systems

Anforderung	Priorität	Quelle (Use Case)
Das System muss einem Host (Performer/Streamer) ermöglichen, eine neue Session (Raum) zu erstellen.	Must	Alle
Das System muss Nutzern den Beitritt via alphanumerischem Raum-Key und QR-Code ermöglichen.	Must	Alle
Das System muss dem Host eine Echtzeit-Liste der verbundenen Teilnehmer anzeigen.	Must	Musik / Stream
Das System muss die Übertragung von Rohdaten des Accelerometers vom Teilnehmer-Gerät an den Host ermöglichen.	Must	Musik
Das System muss (nach Freigabe) den Kamera-Feed von Teilnehmern an den Host übertragen können.	Should	Musik
Das System muss eine Zuweisung (Mapping) von eingehenden Daten (z. B. Sensorwerte) auf Ausgabeparameter (Sound, Visuals) ermöglichen.	Must	Musik
Der Host muss die Logik der Session über einen Node-Graphen bearbeiten können (Hinzufügen von Teilnehmer-Knoten).	Could	Stream
Das System muss Audiosignale vom Host an die Clients senden können.	Should	Musik
Das System muss fähig sein, Vibrationsmuster auf den Endgeräten der Teilnehmer auszulösen.	Should	Musik
Der Host muss das User Interface (GUI) auf den Client-Geräten dynamisch verändern können (z. B. Wechsel von Leer zu Keyboard-Layout).	Must	Stream
Das System muss verschiedene Layouts (z. B. eine Zeichenfläche) auf dem Client bereitstellen können und bei Touch-Eingaben die Koordinaten an den Host senden (Bidirektionalität).	Must	Kunst
Das System muss ermöglichen, dass sich komplexere Logiken und Abläufe implementieren lassen (z. B. Mapping von Zeichnungs-Input auf eine große Leinwand).	Must	Kunst
Das System muss einen Code-Editor oder eine Scripting-Schnittstelle bieten, um Custom-Layouts oder Logiken zu definieren.	Could	Stream
Das System muss externe Hardware-Signale (z. B. MIDI-Pedal, Mikrofon-Pegel) als Input-Parameter akzeptieren.	Could	Musik / Stream

Table 1: Funktionale Anforderungen

Anforderung	Metrik / Akzeptanzkriterium
Die Verzögerung zwischen Nutzer-Interaktion (z. B. Tastendruck, Sensorbewegung) und Audio-Feedback muss minimiert sein, um rhythmisches Musizieren zu ermöglichen.	< 50 ms (idealerweise < 20 ms im LAN)
Um Bandbreitenprobleme zu vermeiden, sollte die Visualisierung bevorzugt client-seitig (auf dem Nutzer-Handy) berechnet werden, statt als Videostream gesendet zu werden.	Datentransfer < Video-Bitrate (z. B. nur JSON-Status-Updates statt H.264-Stream)
Das System muss stabil mit einer hohen Anzahl an Teilnehmern gleichzeitig funktionieren.	Test mit min. X gleichzeitigen Verbindungen
Die Client-Anwendung muss ohne Installation (z. B. als WebApp/PWA) oder als Cross-Platform App auf iOS und Android laufen.	Browser-Kompatibilität / unterstützte OS-Versionen

Table 2: Nicht-funktionale Anforderungen